

Evaluating Retrieval-Augmented Generation Frameworks Under Local Deployment Constraints

A Comparative Study of Single-Retrieval and Corrective Retrieval

Pipelines

Fadi MARDELLI
George-Adelin ANIȚEI

Promotor:

Prof. Dr. Mihai Bâce

Master's Thesis submitted to obtain the degree
of Master of Science in Electronics and ICT
Engineering Technology

Academic Year 2025 - 2026



Evaluating Retrieval-Augmented Generation Frameworks Under Local Deployment Constraints

A Comparative Study of Single-Retrieval and Corrective Retrieval Pipelines

Mardelli Fadi, Aniței George-Adelin

Master in Electronics and ICT , Faculty of Engineering Technology, Campus GROUP T Leuven, Andreas Vesaliusstraat 13, 3000 Leuven, Belgium

Promotor: Prof. Dr. Mihai Bâce

Electronics and ICT , Faculty of Engineering Technology, Campus GROUP T Leuven, Andreas Vesaliusstraat 13, 3000 Leuven, Belgium, <mihai.bace@kuleuven.be>

ABSTRACT

Retrieval-Augmented Generation (RAG) improves language-model outputs by retrieving external evidence before generation. However, corrective RAG systems are often evaluated with cloud-hosted models, external search, or server-class hardware. This thesis compares two pipelines under local deployment constraints: a traditional one-pass RAG pipeline and a bounded corrective RAG pipeline adapted for laptop-local inference. The local evaluation uses Llama and Qwen models in sizes that can run on laptop-class hardware.

The main question is whether the corrective stage provides enough retrieval or answer-quality benefit to justify its additional latency and computational cost. This trade-off matters because one-pass RAG is cheaper and simpler, but it has no mechanism for repairing weak first-pass retrieval. The study also varies model family and size to examine differences in corrective behavior. Larger models are evaluated on cloud-hosted GPU hardware as a deployment reference, showing what is gained or lost when moving from cloud-scale inference to smaller local models that offer privacy, offline access, and lower running cost.

The results show that bounded correction is useful mainly when it redirects retrieval toward evidence missed by the first pass. On LegalBench-mini, Qwen-based correction improves evidence recovery beyond compact and wider one-pass baselines. On HotpotQA, wider one-pass retrieval often matches or exceeds the corrective pipeline, so some gains over the compact baseline are retrieval-depth effects. Corrective activation differs by model family and scale, making the controller a central source of variation. The cloud-scale runs improve HotpotQA answer quality, but do not surpass the best local model on LegalBench-mini evidence recovery. Overall, bounded correction is most justified when missed evidence matters more than response speed.

CONTENTS

1	Introduction	7
2	Related Work	8
2.1	Retrieval-Augmented and Corrective Generation.	8
2.2	Local and Resource-Constrained RAG Deployment	9
2.3	Benchmarks and Evaluation	9
2.4	Positioning of This Work	10
3	Methodology	10
3.1	Benchmarks and Task Settings	10
3.1.1	Benchmark Scope and Comparability	10
3.2	Shared Retrieval Backbone	11
3.3	Pipeline Architectures	11
3.3.1	Traditional RAG	11
3.3.2	Bounded Corrective RAG	11
3.4	Context Construction and Fusion	12
3.5	Task-Specific Corrective Behavior	12
3.5.1	LegalBench-mini	13
3.5.2	HotpotQA	13
3.6	Models and Inference Setup	13
3.7	Evaluation Metrics	13
3.7.1	Retrieval Quality	13
3.7.2	Answer Quality	14
3.7.3	Corrective Behavior	14
3.8	Experimental Protocol	14
3.9	Controlled Latency Evaluation	15
3.10	Reproducibility and Validation Protocol	16
4	Evaluation and Results	16
4.1	Results Overview.	16
4.2	LegalBench-mini Results	16
4.2.1	Overall LegalBench-mini Results	16
4.2.2	Per-Subset LegalBench-mini Results	18

4.3	HotpotQA Results	18
4.3.1	Overall HotpotQA Results	18
4.3.2	Retrieval and Answer Effects of Correction	19
4.4	Retrieval-Width Diagnostics	20
4.4.1	LegalBench-mini	20
4.4.2	HotpotQA	20
4.5	Efficiency and Deployment Cost	21
4.5.1	Latency and Memory	21
4.5.2	Controller Overhead and Sustained-Load Behavior	21
4.5.3	Runtime Reference Across Deployment Settings	21
4.6	Cross-Environment Validation	23
5	Discussion	23
5.1	Corrective RAG as Retrieval Repair	23
5.2	Efficiency-Effectiveness Trade-offs Under Local Deployment	25
5.3	The Controller Is a Central Bottleneck	25
6	Limitations	26
7	Conclusion and future work	26
7.1	Conclusion	26
7.2	Future Work	27
	Appendices	30
A	Hardware and Runtime Configuration	A-1
A.1	Hardware and Operating System	A-1
A.2	Supplementary GPU Pod Configurations	A-1
A.3	Python Environment and Core Libraries	A-1
A.4	Local Generator Models	A-1
A.5	Shared Inference and Retrieval Settings	A-1
B	Dataset and Benchmark Construction.	B-3
B.1	LegalBench-mini Construction	B-3
B.2	Balanced LegalBench-mini Subset for Controlled Studies	B-3
B.3	HotpotQA Closed-Corpus Adaptation	B-3

C	Prompt Templates	C-4
C.1	Shared Answer Prompt Template	C-4
C.2	Final Answer System Prompts	C-4
C.2.1	Legal Answer Prompt	C-4
C.2.2	HotpotQA Answer Prompt	C-4
C.3	Controller Decision Prompts	C-4
C.3.1	Source-Focused Legal Sufficiency Check	C-4
C.3.2	Global Multi-Document Sufficiency Check	C-4
C.4	Corrective Query Planning Prompts	C-5
C.4.1	Legal Rewrite Prompt	C-5
C.4.2	Global Bridge-Planning Prompt	C-5
C.5	Answer Selection Prompt	C-6
D	LegalBench-mini Subset Visualisation	D-7
E	Supplementary Diagnostics and Resource Profiles.	E-8
E.1	Illustrative LegalBench-mini Query Traces	E-8
E.2	Paired Uncertainty Checks	E-10
E.3	Sentence-Level Rescoring for HotpotQA Qwen 7B	E-11
E.4	Qwen 7B Supplementary Resource Profile.	E-11
E.5	Qwen 3B Supplementary Resource Profile.	E-12
E.6	Latency Decomposition	E-13
E.7	Cross-Environment Validation Details	E-13

1 INTRODUCTION

Large language models can generate fluent and useful text, but their knowledge is largely parametric and fixed after training. As a result, they may produce plausible outputs that lack factual support or contradict external evidence, especially in tasks requiring factual accuracy or access to external information Ji et al. (2023).

Retrieval-Augmented Generation (RAG) addresses this limitation by combining document retrieval with generation at inference time. This enables the model to ground outputs in external evidence rather than relying only on memorized knowledge Lewis et al. (2020); Shuster et al. (2021). RAG is therefore useful in tasks that require factual grounding, source traceability, and document-level control Gao et al. (2023).

A common RAG configuration is the one-pass retrieval pipeline, in which a retriever selects a fixed set of passages and the language model generates an answer from that retrieved context Lewis et al. (2020); Gao et al. (2023). This design is efficient and easy to deploy, but it has no explicit mechanism for detecting incomplete retrieval or recovering when relevant evidence is missing. This limitation matters in scenarios that require precise evidence recovery from long documents or the combination of evidence from multiple sources.

Recent work has investigated more adaptive RAG designs, including corrective and agentic workflows. The approach studied here is closest to Corrective Retrieval-Augmented Generation (CRAG), where the system assesses first-pass retrieval and triggers corrective action when the evidence appears insufficient Yan et al. (2024). The pipeline evaluated in this thesis performs an initial retrieval pass and allows at most one additional corrective retrieval stage before generating the final answer. We refer to this as a bounded corrective pipeline.

Unlike more open-ended adaptive or agentic RAG workflows that may repeat retrieval, rewriting, or reasoning until a stopping condition is reached, this pipeline allows only one corrective stage. This fixed bound makes the comparison experimentally interpretable and feasible on laptop-class consumer hardware, because any quality gain can be evaluated against a fixed maximum increase in retrieval calls, model calls, and latency.

Most corrective and adaptive RAG work has been evaluated with API-based models or server-scale infrastructure Gao et al. (2023); Yan et al. (2024). In contrast, fully local deployment changes the evaluation focus from quality alone to quality under data-control and hardware constraints. Documents and prompts remain private in the user’s environment, the system can run offline with-

out network access, and the user maintains full control over the retrieval index, model, and stored data. At the same time, local execution exposes costs that are typically hidden in cloud-based systems, such as limited memory, inference latency, and power consumption. Recent literature treats these privacy, offline-use, latency, memory, and power trade-offs as central to evaluating on-device RAG systems Park et al. (2025).

In this thesis, fully local means that no remote model API is used; retrieval and generation run with open-weight models on laptop-class consumer hardware.

Privacy-sensitive legal applications are a natural use case for local RAG. Legal professionals and small offices often handle confidential material, making local inference attractive for privacy, offline access, and control over data storage. Retrieval-based systems can also support attribution and selective removal of sensitive information more directly than purely parametric models because non-parametric datastores can be updated or removed without retraining the model Min et al. (2024). Professional guidance for the European legal sector similarly highlights risks associated with cloud computing in legal practice and the need for lawyers to assess how cloud services handle professional data Council of Bars and Law Societies of Europe (2025).

Prior adaptive and corrective RAG work shows that retrieval can be evaluated, revised, or routed dynamically, but it does not establish whether one bounded corrective pass is useful when all inference runs locally Yan et al. (2024); Asai et al. (2024); Jeong et al. (2024). Local open-weight deployment is increasingly feasible, this alone does not show whether extra retrieval-control steps are worth their latency and resource cost Park et al. (2025). Existing retrieval and question-answering benchmarks also do not show whether bounded correction behaves similarly across source-focused legal evidence recovery and multi-document question answering Pipitone and Alami (2024); Yang et al. (2018).

We evaluate the pipelines using two complementary benchmarks. The first is **LegalBench-mini**, a compact legal-domain benchmark derived from LegalBench-RAG Pipitone and Alami (2024). We use it as retrieval-focused benchmark to evaluate source-grounded legal evidence recovery, rather than as a full assessment of legal advice quality. The second is **HotpotQA**, evaluated here as a localized, closed-corpus adaptation of the standard multi-hop question answering benchmark Yang et al. (2018). These benchmarks enable comparison of corrective retrieval performance in two structurally different settings: source-focused evidence recovery and multi-document question answering.

The empirical comparison uses a shared local retrieval setup across multiple open-weight local models from the Llama and Qwen families. These options were selected as representative open-weight, instruction-tuned models with variants available in parameter scales suitable for local deployment on a consumer laptop. The 3B-class and 7B/8B-class models provide both a model-family comparison and a small-versus-larger scale comparison of corrective behavior on a consumer laptop Grattafiori et al. (2024); Meta AI (2024); Qwen Team (2024).

To contextualize the local results, we also include larger matched Llama and Qwen models running on a cloud hosted GPU hardware. This provides a deployment reference for interpreting the trade-off of moving from big models that run on the cloud and models that can run locally on a laptop.

Research Questions.

- **RQ1:** Under local hardware constraints, when does bounded corrective RAG improve over one-pass RAG, and at what cost?
- **RQ2:** How does task structure affect the value of bounded correction in legal evidence recovery and multi-document question answering?
- **RQ3:** How do model family, model scale, and deployment setting affect corrective RAG performance and cost?

Contributions. The contributions of this thesis are as follows:

- A controlled empirical comparison of *traditional* and *bounded corrective* RAG in a fully local inference setting.
- An evaluation across two structurally different benchmark types: source-focused legal evidence recovery on LegalBench-mini and multi-document question answering on HotpotQA.
- A controlled cross-model comparison of Llama and Qwen variants that reports how corrective activation, retrieval quality, answer quality, latency, and memory usage change across model family and scale.
- A deployment-oriented analysis of local feasibility, including latency, memory usage, energy estimates, cross-environment validation, and a cloud-scale reference with larger matched open-weight models.

Summary of Findings. The results show that bounded corrective RAG improves performance only in some settings. It improves legal evidence retrieval with Qwen models, but this pattern does not hold consistently for Hot-

potQA in the closed-corpus setup used here. In that setting, wider one-pass retrieval often matches or exceeds the corrective pipeline, indicating that some gains over the $k = 5$ baseline come from retrieving more candidates rather than from correction itself. The main cost of correction is the extra local model calls used for control, rewriting, generation, and answer selection. The cloud-scale runs improve HotpotQA answer results and change corrective activation patterns, but they do not surpass the best local Qwen model on LegalBench-mini evidence recovery.

2 RELATED WORK

In this chapter we review the most relevant work to our study: retrieval-augmented generation, adaptive and corrective retrieval, local deployment constraints, and benchmarks for evaluating retrieval-grounded systems.

2.1 Retrieval-Augmented and Corrective Generation

Retrieval-Augmented Generation (RAG) combines document retrieval with sequence generation at inference time, allowing a language model to condition its answer on external, non-parametric evidence rather than relying only on fixed parametric knowledge Lewis et al. (2020). This makes RAG suitable for settings where factual grounding, evidence traceability, and access to domain-specific documents are important. Recent surveys describe RAG as a broader design space involving retrieval, augmentation, and generation choices, rather than as a single fixed architecture Gao et al. (2023).

RAG systems can use sparse lexical retrieval, dense vector retrieval, or hybrid approaches. Dense retrieval encodes queries and passages into a shared embedding space and ranks passages by vector similarity. Dense Passage Retrieval (DPR) established dual-encoder retrieval for open-domain question answering Karpukhin et al. (2020), while Sentence-BERT supports semantically meaningful sentence embeddings for similarity search Reimers and Gurevych (2019). Facebook AI Similarity Search (FAISS) provides efficient nearest-neighbor search over dense vectors Johnson et al. (2019). This thesis uses dense retrieval over a FAISS index with a local sentence-embedding model as the shared retrieval backbone for both pipelines.

The work most directly related to this thesis studies how retrieval can be made adaptive or corrective. FLARE performs active retrieval during generation using predicted future content and low-confidence tokens Jiang et al. (2023). Self-RAG trains a model to retrieve and critique using reflection tokens Asai et al. (2024). Adaptive-RAG se-

lects retrieval strategies according to question complexity Jeong et al. (2024). For multi-step question answering, IR-CoT interleaves chain-of-thought reasoning with retrieval Trivedi et al. (2023).

CRAG is closest to this thesis because it evaluates first-pass retrieval and triggers corrective action when retrieved evidence appears weak Yan et al. (2024). The pipeline evaluated here is inspired by this corrective idea, but it is not a direct reproduction of the original CRAG system. Original CRAG uses a separate fine-tuned retrieval evaluator, confidence-based correction actions, web search, and knowledge refinement over retrieved content. In contrast, this thesis evaluates a closed-corpus local adaptation: no external web search is used, correction is limited to at most one additional retrieval stage, and the same local model performs sufficiency checking, query rewriting, generation, and HotpotQA answer selection. The contribution is not a new CRAG architecture, but an evaluation of whether this locally feasible corrective design provides enough benefit to justify its cost.

These methods show that retrieval control can improve robustness, but they also introduce stronger assumptions or more complex control mechanisms than the bounded design studied here. Some rely on trained reflection behavior, some use routing or iterative generation-time retrieval, and some allow multiple retrieval-reasoning steps. This thesis follows the corrective direction, but limits correction to at most one additional retrieval stage. The question becomes narrower: whether one additional corrective retrieval stage still improves answer quality enough to justify its local inference cost.

In both CRAG and the bounded corrective pipeline implemented in this study, correction primarily targets the retrieval and context-construction stage rather than post-hoc answer editing.

Query formulation is also relevant because the corrective pipeline can rewrite the original query before retrying retrieval. Query rewriting has been shown to improve retrieval-augmented question answering by reformulating the user question into a query better suited to retrieval and reading Ma et al. (2023). Step-back prompting is related because it uses abstraction to derive higher-level concepts from specific questions, which can be useful for bridge-style reasoning Zheng et al. (2024). This thesis treats query rewriting as one component of the bounded corrective pipeline, not as a separately optimized retriever.

2.2 Local and Resource-Constrained RAG Deployment

In a local setting, retrieval quality is not the only concern. Extra retrieval and control steps also have a direct cost in latency and resource use. This matters for corrective RAG because a corrective stage can add a controller decision, query rewriting, additional retrieval calls, answer selection, and a larger final context. The cost of correction is therefore not only a retrieval cost, but also an inference cost.

Local LLM deployment has become more practical through open-weight models and post-training quantization. GPTQ and AWQ reduce memory requirements through low-bit weight quantization while aiming to preserve model quality Frantar et al. (2023); Lin et al. (2024). Runtime systems such as llama.cpp support local inference with quantized open-weight models and use GGUF as the model storage format Gerganov et al. (2023). These developments make local testing of multi-billion-parameter instruction-tuned models feasible without external API access.

However, local and edge deployment impose constraints that are less visible in server-scale evaluations. Work on edge and local LLM deployment identifies memory capacity, computation, latency, and energy use as important constraints when inference is moved outside centralized infrastructure Wang et al. (2025); Tian et al. (2025). Recent on-device RAG work treats latency, memory usage, and power consumption as first-class evaluation criteria Park et al. (2025). This thesis follows that deployment-oriented view, while focusing on the costs measured in the experiments: latency, peak resident memory, retrieval-call counts, prompt and context token pressure, sustained-load latency drift, and supplementary estimated power and energy per query.

Long-context inference is another alternative to retrieval-heavy designs, but it is not a universal replacement. Models can use relevant evidence less effectively when it appears in the middle of a long prompt Liu et al. (2024), and retrieval can remain complementary to long context while requiring less computation than simply expanding the context window Xu et al. (2024). This supports the deployment question addressed here: do additional retrieval-control steps improve results enough to justify their local cost?

2.3 Benchmarks and Evaluation

RAG evaluation needs to distinguish retrieval quality from final answer quality. KILT helped standardize knowledge-intensive evaluation by combining multiple tasks with provenance-aware evaluation Petroni et al. (2021). More

recently, BERGEN argued for reproducible end-to-end RAG benchmarking by standardizing comparisons across retrievers, rerankers, generators, datasets, and metrics Rau et al. (2024). This matters because better retrieval does not always produce better answers if the generator fails to use the evidence effectively.

The legal benchmark used in this thesis is derived from LegalBench-RAG. LegalBench evaluates legal reasoning with language models, but it is not primarily designed to isolate retrieval quality Guha et al. (2023). LegalBench-RAG addresses this more directly by focusing on retrieval for legal RAG pipelines and linking legal questions to relevant source-document spans Pipitone and Alami (2024). For this reason, we use LegalBench-mini as a retrieval-focused legal evidence benchmark and not as a quality evaluation of legal answers.

For multi-document question answering, HotpotQA is relevant because it combines answer supervision with supporting-fact supervision, allowing both retrieval and end-to-end answer quality to be examined Yang et al. (2018). This thesis uses HotpotQA in a local closed-corpus setting to study whether improved retrieval coverage translates into improved answers in multi-document question answering.

These benchmarks provide the task context for this thesis, but their public results are not used as direct baselines because this work uses a local closed-corpus setup, a shared dense retriever, and a bounded corrective pipeline.

2.4 Positioning of This Work

This thesis contributes a controlled empirical comparison of traditional one-pass RAG and bounded corrective RAG under fully local deployment constraints. It studies whether one additional corrective stage adds value beyond one-pass retrieval, and whether that value is large enough to justify the added local latency and resource cost.

The work is closest to CRAG-style corrective retrieval, but it studies a narrower setting: one bounded corrective stage, no web search, no trained reflection model, and no open-ended multi-step retrieval. Prior adaptive RAG work motivates retrieval control; this thesis tests whether that control remains worthwhile when every additional model call runs locally.

The contribution is not a new RAG architecture, but instead an evaluation of when bounded corrective RAG helps, when a stronger one-pass baseline is sufficient, and what local costs the corrective step imposes.

3 METHODOLOGY

We compare two RAG pipeline designs that share the same local dense-retrieval backbone: traditional one-pass RAG and bounded corrective RAG. This keeps the comparison at the pipeline level rather than isolating correction as a single component. We focus on configurations that remain practical for repeated laptop-local runs.

The complete hardware, software, and inference settings are documented in Appendix A. The following sections describe the benchmarks, retrieval backbone, pipeline designs, evaluation metrics, and experimental protocol.

3.1 Benchmarks and Task Settings

LegalBench-mini is derived from LegalBench-RAG Pipitone and Alami (2024). It is a retrieval-focused legal benchmark in which success is defined by the final retrieved context containing annotated supporting evidence. It contains 776 cases divided evenly across four subsets: PrivacyQA, ContractNLI, MAUD, and CUAD, with 194 cases per subset. Each case pairs a legal query with annotated supporting evidence linked to source documents.

HotpotQA Yang et al. (2018) is a multi-document question answering benchmark that supports both retrieval evaluation and end-to-end answer evaluation. The original dataset includes easy, medium, and hard difficulty levels, but the held-out distractor validation split used here contains only hard difficulty cases. Each case includes a question, a gold answer, and gold supporting titles. The validation set contains 7,405 cases, and all comparative experiments use the same fixed first 1,000-case slice. Since this is a closed local corpus rather than the original full-wiki setting, the scores are intended for within-study comparison rather than direct comparison with published results. The validation split contains two question types: bridge questions and comparison questions. The fixed 1,000-case slice preserves this question-type balance, with 807 bridge and 193 comparison questions, close to the full validation split distribution of 5,918 bridge and 1,487 comparison questions.

Figure 3.1 shows representative questions from each LegalBench-mini subset and from the two HotpotQA question types. Further details of the benchmark construction are provided in Appendix B.

3.1.1 Benchmark Scope and Comparability

This thesis does not aim to compare with state-of-the-art results. Because full benchmark evaluation requires

Panel A: LegalBench-mini example questions

PrivacyQA	Consider “Fiverr’s privacy policy; who can see my information?”
ContractNLI	Consider Motorola’s Non-Disclosure Agreement; Does the document restrict the use of Confidential Information to the purposes stated in the Agreement?
MAUD	Consider the Merger Agreement between “Avan-grid, Inc.” and “PNM Resources, Inc.”; What is the Type of Consideration
CUAD	Consider the Cooperation Agreement between HPIL ENERGYTECH Inc. and GINARES GROUP AG for Renewable Energy Projects; Can this contract be terminated for convenience, and under what conditions?

Panel B: HotpotQA example questions

Comparison	Were Scott Derrickson and Ed Wood of the same nationality?
Bridge	What government position was held by the woman who portrayed Corliss Archer in the film Kiss and Tell?

Figure 3.1: PrivacyQA asks about privacy-policy content, ContractNLI asks whether a contract entails a yes/no claim about its contents. MAUD asks about M&A agreement deal points, including clauses, definitions, and deal terms. CUAD asks about clause-level information in commercial contracts. HotpotQA comparison questions compare two entities, while bridge questions require finding one fact through another.

substantial runtime on the laptop, the experiments use locally adapted benchmark settings and a fixed HotpotQA slice. The public benchmarks use different corpus, pre-processing, and evaluation settings, hence the results are interpreted as within-study comparisons rather than direct comparisons with published benchmark scores.

The relevant reference points are therefore within-study baselines and deployment references. Traditional $k = 5$ is the compact one-pass baseline used in the main comparison, while traditional $k = 20$ tests whether corrective gains can be recovered by a wider one-pass retrieval baseline. The cloud-scale comparison contextualizes the trade-off between laptop-feasible local models and larger models that require server-class GPU hardware.

3.2 Shared Retrieval Backbone

For retrieval, both systems use a shared dense-vector search backbone as a local baseline that remains feasible to build and run repeatedly on laptop-class hardware. Fixing retrieval keeps the comparison focused on pipeline behavior, while the corrective pipeline adds controller decisions, query rewriting, optional backup retrieval, fusion, and task-specific context selection.

Each retrieved unit is a *chunk*, that is, a limited text segment produced by the chunking procedure and stored in the retrieval index. Chunks are embedded with sentence-transformers Reimers and Gurevych (2019) using BAAI/bge-base-en-v1.5, normalized at encoding time, and indexed with Facebook AI Similarity Search (FAISS) Johnson et al. (2019) using a flat inner-product index. This embedding setup is used for every benchmark and model condition. Documents are chunked into

800-character segments with 150-character overlap. The segmentation is aware of text boundaries. Structured documents such as legal contracts are split on line boundaries, while prose is split on sentence boundaries. This preserves the clause and sentence units that matter for evidence recovery.

3.3 Pipeline Architectures

Figure 3.2 illustrates the architecture of both implemented pipelines.

3.3.1 Traditional RAG

When the traditional pipeline is given a question, it retrieves chunks from the benchmark-specific index, packs them into a bounded prompt context, and generates the final output.

3.3.2 Bounded Corrective RAG

The bounded corrective pipeline begins with the same first-pass retrieval and context assembly. Before generation, a task-specific controller decides whether the first-pass context appears sufficient. If it is not, it triggers one corrective stage where the same local model generates one primary rewritten query and optionally one backup query. Finally, retrieved chunks from the corrective stage are fused with first-pass chunks before final context construction.

We use the same local model for control, query rewriting, answer generation, and in the case of HotpotQA, answer

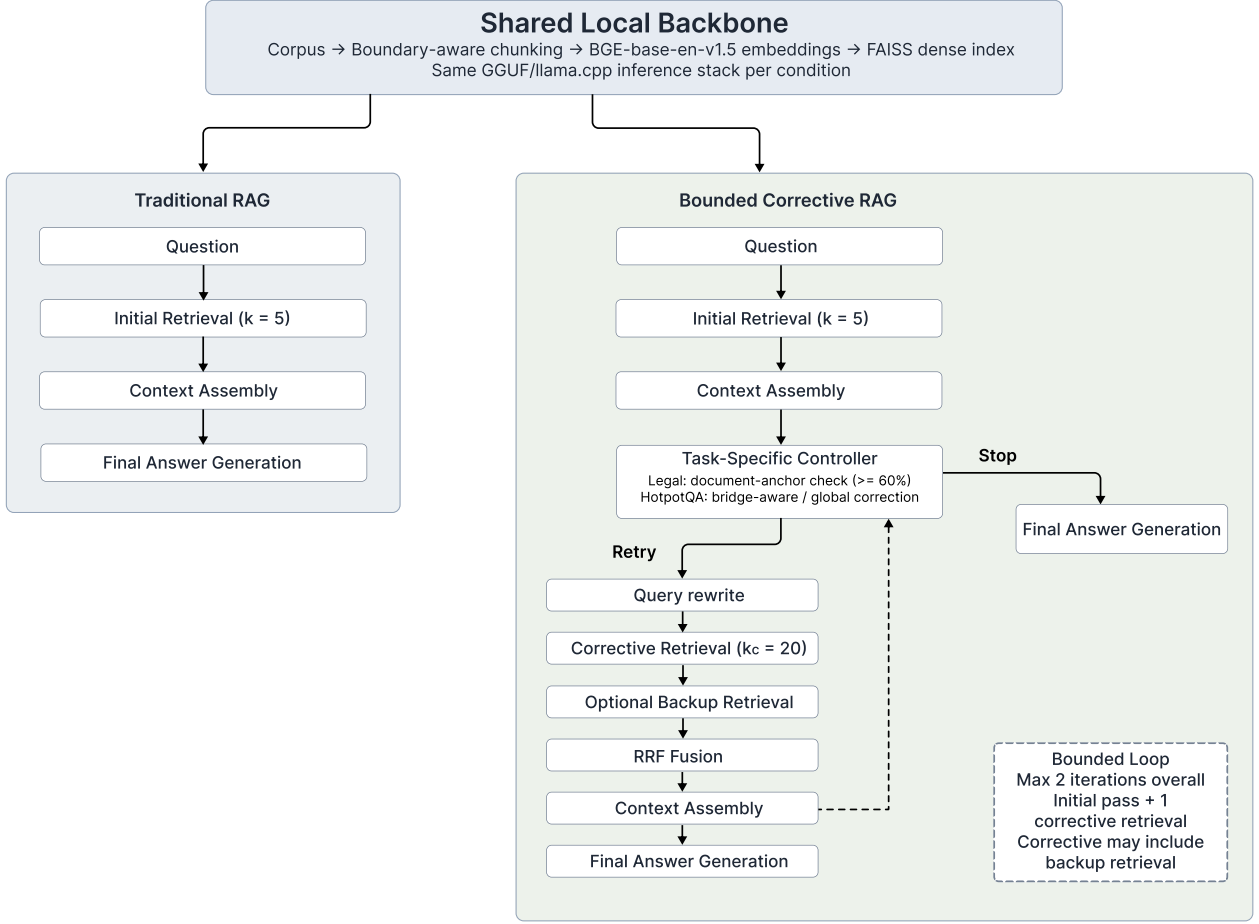


Figure 3.2: Architecture comparison of traditional RAG (left) and bounded corrective RAG (right). The corrective pipeline adds a task-specific controller and at most one additional retrieval stage.

selection as well. When HotpotQA correction runs, the system generates both a first-pass and corrected-pass answer, then selects the one that is more supported by its context.

We limit the implementation to only one additional corrective stage because, on the laptop used for experiments, extra retrieval-generation cycles increase runtime and thermal load substantially during our testing.

3.4 Context Construction and Fusion

We distinguish the retrieved candidate set from the final used context, since later metrics rely on both. The *retrieved candidate set* is the ranked list of chunks returned before context-budget trimming. The *final used context* is the subset of those chunks that is actually inserted into the generation prompt.

In both pipelines, the initial retrieval pass returns the top $k = 5$ chunks ordered by relevance. In the traditional pipeline these are used directly. In the bounded correc-

tive pipeline, if the corrective pass stage is activated, it retrieves up to $k_c = 20$ additional candidates. The two result sets are then merged using reciprocal rank fusion (RRF) Cormack et al. (2009). RRF assigns higher scores to chunks ranked highly in either pass and further boosts scores of chunks retrieved in both passes.

Context assembly is identical in both pipelines. Chunks are concatenated in ranked order as raw text, separated by blank lines, until a shared 6,000-character budget is reached. If the next chunk would exceed the 6,000-character budget, it is not included. The budget is set to cover the full ground-truth evidence in all questions across both benchmarks while avoiding unnecessarily long contexts, which have been shown to degrade how reliably models use embedded evidence Liu et al. (2024).

3.5 Task-Specific Corrective Behavior

The bounded corrective pipeline has the same high-level structure across benchmarks, but the corrective stage is

task-specific.

3.5.1 LegalBench-mini

Many LegalBench-mini queries refer to a source document such as an agreement, policy, or contract. The system first checks the question text for source-specific cues such as `agreement`, `document`, or `policy`. If such a cue is found, it estimates a dominant source document from the top five first-pass chunks. If at least 60% of those chunks come from the same document, the anchor is treated as reliable and corrective retrieval is restricted to that document. Otherwise, the system falls back to global retrieval.

For source-specific questions, the controller either stops or triggers a retry. On retry, the model generates a primary query and, when available, a backup query to search for the missing evidence. If no reliable source document can be identified, the retry uses global retrieval instead of document-restricted retrieval.

3.5.2 HotpotQA

HotpotQA questions require evidence across multiple documents, so corrective retrieval is not restricted to one source document. The controller identifies missing information, such as a bridge fact, second entity, or comparison target, and generates a primary step-back query with an optional backup query. If no missing information is detected, correction is skipped.

When correction is activated, the new results are fused with the first-pass chunks. The system then generates both a first-pass and corrected-pass answer and selects the answer more directly supported by its context. If the selector chooses the first-pass answer, the final used context recorded for evaluation is the first-pass context rather than the corrected fused context.

Prompt templates for controller, rewriting, generation, and answer-selection steps are provided in Appendix C.

3.6 Models and Inference Setup

Table 3.1 lists the models used. Other open-weight families, such as Gemma, Mistral, or Phi, could also have been used. We chose Llama and Qwen families since they provide a controlled comparison across two model families and two feasible local size bands while keeping all conditions runnable in the same quantized GGUF setup on one laptop. All models run as `Q4_K_M` GGUF checkpoints through `llama.cpp` Gerganov et al. (2023) on a MacBook Air M4 (10-core CPU, 10-core GPU, 16 GB uni-

fied memory) with GPU-accelerated inference via Apple Silicon Metal.

For the cloud-scale reference, the same implementation is used with larger matched GGUF checkpoints from the same two model families: Qwen 2.5 72B Instruct and Llama 3.1 70B Instruct. These runs keep the retrieval index, context construction, prompts, and evaluation metrics aligned with the local runs, while changing model scale and execution hardware.

All runs use the same runtime context limit of 4,096 tokens, retrieved-context budget of 6,000 characters, seed of 42, and temperature of 0.0.

Appendix A, Table A.1, lists the full runtime settings.

3.7 Evaluation Metrics

We report retrieval, answer-quality, and corrective-behavior metrics. All reported effectiveness metrics are aggregated at the case level over the benchmark slice for each model pipeline condition. Binary metrics are reported as proportions of successful cases, and per-case numeric metrics are reported as means.

3.7.1 Retrieval Quality

For LegalBench-mini, retrieval quality is evaluated against the annotated evidence spans. A case is counted as a retrieval success when the target set contains at least one retrieved chunk from the correct source document whose character range overlaps a gold evidence span, that is, when `chunk_start < gold_end` and `chunk_end > gold_start`. **Evidence recall** applies this test to the final used context; **top- k hit rate** applies the same test to the retrieved candidate set before context trimming. In the traditional first-pass $k=5$ setting the 6,000-character budget did not trim the retrieved list, so this distinction matters mainly in corrective or wider-retrieval settings.

For HotpotQA, retrieval quality is title-based. Each retrieved chunk carries the title of its source document, and a gold supporting title is counted as recovered when a retrieved chunk comes from a document with the same title. No additional title normalization is applied. **Supporting-title recall** is computed per case as the fraction of gold titles recovered in the final used context and then averaged over cases. **Top- k hit rate** counts whether at least one gold title appears in the candidate set before trimming. **Strict evidence recall** counts whether all gold titles appear in the final used context. These title-based metrics are coarser than the original HotpotQA sentence-level

Table 3.1: Open-weight generator models used in the evaluation. The first block contains the laptop-local models used in the main factorial comparison; the second block contains the larger cloud-scale reference models. All models are instruction-tuned and run as 4-bit GGUF checkpoints through `llama.cpp`.

Model	Family	Params	HuggingFace ID	Quant.	Ctx.
<i>Laptop-local main models</i>					
Llama 3.2 3B Instruct	Llama 3.2	3 B	meta-llama/Llama-3.2-3B-Instruct	Q4_K_M	4096
Llama 3.1 8B Instruct	Llama 3.1	8 B	meta-llama/Meta-Llama-3.1-8B-Instruct	Q4_K_M	4096
Qwen 2.5 3B Instruct	Qwen 2.5	3 B	Qwen/Qwen2.5-3B-Instruct	Q4_K_M	4096
Qwen 2.5 7B Instruct	Qwen 2.5	7 B	Qwen/Qwen2.5-7B-Instruct	Q4_K_M	4096
<i>Cloud-scale reference models</i>					
Llama 3.1 70B Instruct	Llama 3.1	70 B	meta-llama/Meta-Llama-3.1-70B-Instruct	Q4_K_M	4096
Qwen 2.5 72B Instruct	Qwen 2.5	72 B	Qwen/Qwen2.5-72B-Instruct	Q4_K_M	4096

supporting-fact evaluation, but they match the local adaptation used here, which preserves supporting titles as the stable evidence target.

Table 3.2 summarizes all five retrieval metrics. In the table, N denotes the total number of cases, n a count of cases satisfying the stated condition, and $\bar{r}$ the mean per-case ratio.

3.7.2 Answer Quality

End-to-end answer quality is reported only for HotpotQA. **Answer accuracy** is measured as Exact Match (EM): the predicted answer is counted as correct if it matches the gold answer after lowercasing, removing punctuation and articles, and normalizing whitespace:

$$\text{answer accuracy (EM)} = \frac{\text{exact-match answers}}{\text{total cases}}$$

Answer F1 is computed from token overlap after the same normalization and then averaged over cases. No alias mapping is applied.

In bounded corrective HotpotQA runs, answer quality is evaluated only on the final answer selected by the answer-selection step. For LegalBench-mini, no end-to-end answer metric is reported because the benchmark does not provide gold answer strings for short-answer comparison.

3.7.3 Corrective Behavior

For bounded corrective runs, the following metrics quantify how often correction activates and how much extra retrieval work is performed; for traditional one-pass runs, they take their trivial baseline values. **Retry rate** counts how often a query proceeds to a second retrieval-control iteration:

$$\text{retry rate} = \frac{\text{queries with a corrective retry}}{\text{total queries}}$$

Iterations mean is the average number of retrieval-control iterations per query:

$$\text{iterations mean} = \frac{\text{total retrieval-control iterations}}{\text{total queries}}$$

Each query has at most two iterations: the initial pass and, if activated, one corrective pass. **Retrieval calls mean** is the average number of actual retriever invocations per query:

$$\text{retrieval calls mean} = \frac{\text{total retriever calls}}{\text{total queries}}$$

This can exceed iterations because one corrective pass may issue both a primary query and an optional backup query. Table 3.3 summarizes this distinction.

3.8 Experimental Protocol

We evaluate both pipelines with Llama 3.2 3B, Llama 3.1 8B, Qwen 2.5 3B, and Qwen 2.5 7B on both benchmarks, giving eight conditions per benchmark and sixteen main runs in total.

For HotpotQA, all comparative runs use the fixed first 1,000 validation cases. Using the same held-out slice for every condition avoids introducing additional sampling variation into the main comparison.

Local runs are split into batches to limit sustained thermal load on the laptop, and batch outputs are concatenated before computing final metrics. We do not repeat the full benchmark runs because corrective runs require substantial local runtime. Instead, the repeated controlled-latency sweeps provide a limited fixed-workload repeatability check under the same model files, prompts, retrieval setup, seed, and temperature-zero decoding.

Table 3.2: Retrieval quality metrics. N : total cases; n : cases satisfying the condition; $\bar{r}$: mean per-case ratio.

Metric	Formula	Condition counted	Benchmark	Evaluated on
Evidence recall	n/N	Gold-source chunk overlaps gold span	LegalBench-mini	Final used context
Top- k hit rate	n/N	Gold-source chunk overlaps gold span	LegalBench-mini	Candidate set
Supporting-title recall	$\bar{r}$	Fraction of gold titles recovered per case	HotpotQA	Final used context
Top- k hit rate	n/N	At least one gold title retrieved	HotpotQA	Candidate set
Strict evidence recall	n/N	All gold titles recovered	HotpotQA	Final used context

Table 3.3: Interpretation of iterations and retrieval calls in the bounded corrective pipeline.

Case	Iterations	Retrieval calls
Traditional one-pass run	1	1
Corrective pipeline, no retry	1	1
Corrective pipeline, retry with primary query only	2	2
Corrective pipeline, retry with primary and backup queries	2	3

To test whether bounded-corrective gains can be explained by retrieval width alone, we include targeted Qwen 7B diagnostics. Traditional $k = 20$ runs use the same one-pass pipeline as $k = 5$ with retrieval width increased to $k = 20$.

For LegalBench-mini, an additional bounded corrective diagnostic reduces corrective width from $k_c = 20$ to $k_c = 5$ while keeping controller behavior, routing, query rewriting, fusion, and context construction unchanged. These diagnostics are targeted ablations, not a full additional experimental factor across all models.

After the main local comparison, we run a cloud-scale deployment reference with Qwen 2.5 72B Instruct and Llama 3.1 70B Instruct on H200 GPU hardware. For HotpotQA, both traditional and corrective cloud-scale rows are reported because answer metrics depend on the generator. For LegalBench-mini, only corrective cloud-scale rows are reported because traditional retrieval metrics are determined by the shared retriever and context construction rather than by model size.

Two unrecorded warm-up queries run before each batch to avoid cold-start effects on model loading and memory state. Hardware-sensitive performance results are analyzed separately in the controlled latency study.

Supplementary paired uncertainty checks are reported in Appendix E. Binary metrics use exact two-sided McNemar tests and paired bootstrap percentile confidence intervals; mean-valued HotpotQA metrics use paired bootstrap percentile confidence intervals with 20,000 resamples (seed 42).

3.9 Controlled Latency Evaluation

A separate controlled latency study estimates runtime and memory cost under fixed local conditions. Full continuous benchmark runs can be affected by heterogeneous batch structure and thermal accumulation, which can introduce latency drift between earlier and later queries. The controlled study therefore uses fixed 100-query workloads, repeated sweeps, and stable local execution conditions.

For LegalBench-mini, the workload is a balanced 100-case manifest with 25 cases from each subset, derived with seed 42. For HotpotQA, it is a fixed random 100-case sample from the same first 1,000-case slice, also using seed 42. The same manifest is reused for every model and pipeline condition so that latency differences reflect system behavior rather than changing query mixes.

Each sweep runs the 100-query workload in 10 batches of 10 queries. Two unrecorded warm-up queries are executed before each batch, and a 300-second cooldown is applied between batches. Two pooled sweeps are run per condition: Sweep A uses the canonical manifest order and Sweep B uses the reversed order, so that observed latency drift is less likely to reflect one fixed query ordering.

The reported median latency is the median of the two sweep-level medians, and the reported 95th-percentile latency is the median of the two sweep-level 95th percentiles. Peak resident memory is measured at the process level rather than at the whole-system level. During each query, the benchmark process’s RSS is sampled periodically and the maximum observed value is recorded; each sweep then keeps the largest per-query peak, and the final reported value is the maximum of the two sweep peaks, converted to GiB. RSS measures the peak amount of RAM occupied by the evaluation process during a run, so it indicates the memory footprint required for local execution. This is directly relevant to laptop-class deployment, although in practice RSS is dominated mainly by the loaded model weights and is therefore most informative across model scales. Conditions whose sweep medians differ by more than 15% of their midpoint receive a stability warning.

The controlled latency runs were executed on external power with the laptop lid open, in an idle background-load state with no applications running. Primary efficiency metrics are median latency, 95th-percentile latency, and peak resident memory. Secondary metrics include retrieval latency, generation latency, retrieval calls, iterations, token counts, timeout rate, error rate, and batch drift.

3.10 Reproducibility and Validation Protocol

Prior work shows that LLM outputs can change across hardware and runtime configurations because small numerical differences can propagate into different generated responses Yuan et al. (2025). We therefore reran matched benchmark conditions on a supplementary on-demand GPU pod rented through the RunPod cloud service, configured with one NVIDIA GeForce RTX 4090 GPU, 16 vCPUs, and 62 GB RAM. This allows us to check whether the main effectiveness trends in this study are stable across environments rather than artifacts of one local machine.

Only effectiveness and corrective-behavior metrics are compared in this validation step. Latency, throughput, and memory are excluded because they are hardware-dependent and are handled separately.

4 EVALUATION AND RESULTS

4.1 Results Overview

This chapter reports the results in relation to the three research questions from Chapter 1. The main pipeline comparison addresses RQ1 by comparing traditional one-pass RAG and bounded corrective RAG with respect to retrieval quality, answer quality, and efficiency. The benchmark-level analysis addresses RQ2 by testing whether this trade-off differs between LegalBench-mini and HotpotQA. The model-level and controller-activation analyses address RQ3 by comparing Llama and Qwen variants in retry behavior, quality gains, and computational cost. The $k = 20$ diagnostic further supports RQ1 by testing whether correction gains can be recovered by a stronger one-pass retrieval baseline.

4.2 LegalBench-mini Results

4.2.1 Overall LegalBench-mini Results

Table 4.1 lists the local LegalBench-mini runs and the corrective reference rows from the cloud-scale models. The cloud-scale rows are reported only for corrective mode because, in the traditional LegalBench-mini pipeline, retrieval

scores are independent of model size.

Across the local traditional runs, evidence recall and top- k hit rate are both 0.308. Under bounded corrective RAG, Qwen 3B increases evidence recall to 0.445 and top- k hit rate to 0.562. Qwen 7B reaches the strongest overall LegalBench-mini result, with evidence recall 0.519 and top- k hit rate 0.647.

The local Llama corrective results remain identical to the traditional baseline, with evidence recall 0.308, top- k hit rate 0.308, and retry rate 0.000. In the cloud-scale reference results, Qwen 72B reaches evidence recall 0.441 and top- k hit rate 0.549, while Llama 70B reaches evidence recall 0.488 and top- k hit rate 0.620.

Figure 4.1 compares the corrective values across the local and cloud-scale model tiers. It reports evidence recall, top- k hit rate, and retry rate; retrieval calls mean is kept in Table 4.1 as the direct measure of retrieval-side work.

For the local traditional-versus-corrective comparisons, the paired uncertainty checks in Appendix E present evidence-recall gains for both Qwen models: Qwen 3B has $\Delta = +0.137$, 95% CI $[+0.106, +0.168]$, $p < 0.001$, and Qwen 7B has $\Delta = +0.211$, 95% CI $[+0.175, +0.247]$, $p < 0.001$. Both local Llama comparisons remain unchanged.

Table 4.1: Overall LegalBench-mini results using the Section 3.7 metrics.

Condition	Evidence Recall	Top- <i>k</i> Hit Rate	Retry Rate	Retrieval Calls Mean
<i>Local laptop-feasible runs</i>				
Qwen 3B Traditional	0.308	0.308	0.000	1.000
Qwen 3B Corrective	0.445	0.562	0.937	2.871
Qwen 7B Traditional	0.308	0.308	0.000	1.000
Qwen 7B Corrective	0.519	0.647	0.955	2.907
Llama 3B Traditional	0.308	0.308	0.000	1.000
Llama 3B Corrective	0.308	0.308	0.000	1.000
Llama 8B Traditional	0.308	0.308	0.000	1.000
Llama 8B Corrective	0.308	0.308	0.000	1.000
<i>Cloud-scale reference</i>				
Qwen 72B Corrective	0.441	0.549	0.657	2.312
Llama 70B Corrective	0.488	0.620	0.661	2.320

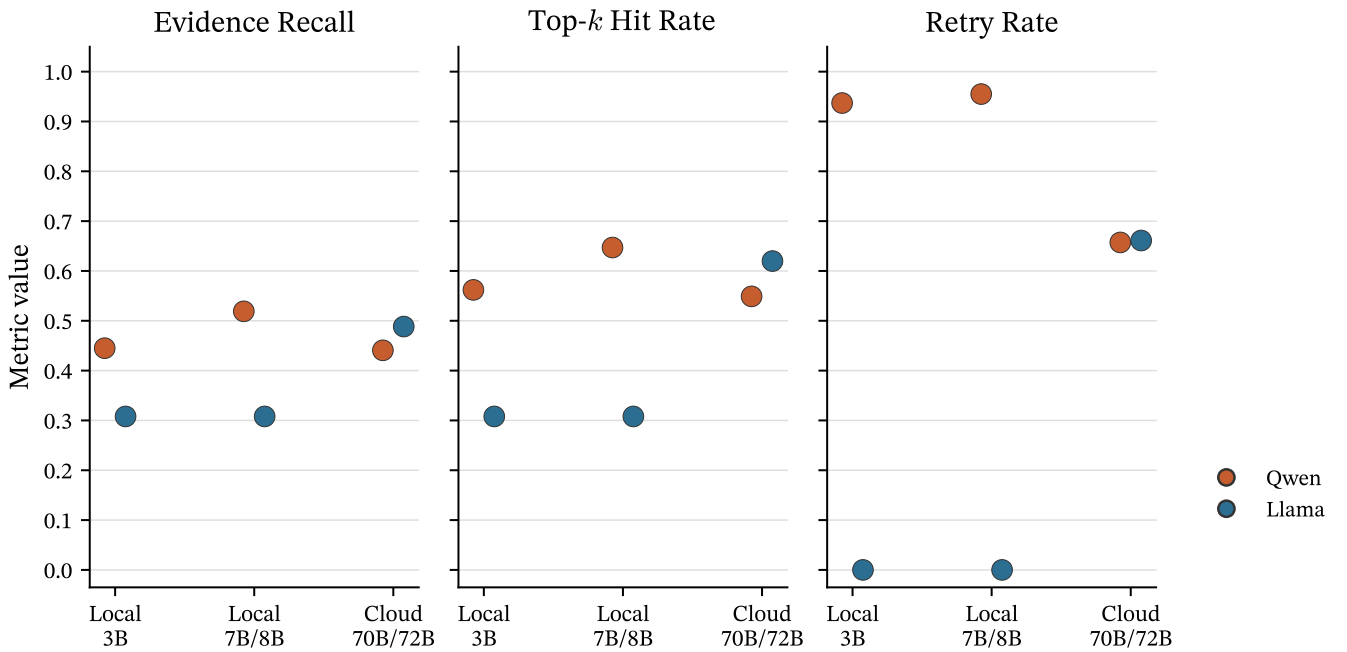


Figure 4.1: LegalBench-mini corrective pipeline results across local and cloud-scale model tiers.

4.2.2 Per-Subset LegalBench-mini Results

Table 4.2 lists local LegalBench-mini evidence recall by subset.

Qwen 3B improves PrivacyQA, MAUD, and CUAD while leaving ContractNLI unchanged. Qwen 7B improves PrivacyQA, MAUD, and CUAD, with increases of +0.165 on PrivacyQA, +0.330 on MAUD, and +0.418 on CUAD.

ContractNLI is the only subset that does not improve. Qwen 3B remains at 0.335, while Qwen 7B decreases from 0.335 to 0.268. A case-level comparison for Qwen 7B shows 21 regressions, where traditional RAG retrieves the correct evidence but bounded corrective RAG does not, compared with 8 cases in the opposite direction. Among the 183 ContractNLI cases where Qwen 7B triggered RETRY, first-pass evidence recall was 0.322 and final evidence recall was 0.251.

For both local Llama models, the corrective and traditional values are identical across all four subsets.

Illustrative recovery and regression traces are provided in Appendix E, Section E.1.

Table 4.2: Per-subset LegalBench-mini evidence recall.

Condition	Privacy QA	Contract NLI	MAUD	CUAD
Qwen 3B Traditional	0.639	0.335	0.046	0.211
Qwen 3B Corrective	0.763	0.335	0.227	0.454
Qwen 7B Traditional	0.639	0.335	0.046	0.211
Qwen 7B Corrective	0.804	0.268	0.376	0.629
Llama 3B Traditional	0.639	0.335	0.046	0.211
Llama 3B Corrective	0.639	0.335	0.046	0.211
Llama 8B Traditional	0.639	0.335	0.046	0.211
Llama 8B Corrective	0.639	0.335	0.046	0.211

4.3 HotpotQA Results

4.3.1 Overall HotpotQA Results

Table 4.3 shows the local HotpotQA runs and the cloud-scale reference results. Figure 4.2 compares the corrective values across the local and cloud-scale model tiers for answer F1, strict evidence recall, and retry rate. Among the local conditions, Qwen 7B Corrective has the strongest HotpotQA scores: answer accuracy 0.465, answer F1 0.594, supporting-title recall 0.853, and strict evidence recall 0.725. Qwen 3B changes only slightly under correction, from answer F1 0.512 to 0.513 and strict evidence recall 0.638 to 0.640.

For the local Llama models, Llama 3B remains at answer accuracy 0.364, answer F1 0.508, and strict evidence recall 0.638. Llama 8B increases strict evidence recall from 0.638 to 0.674, answer accuracy from 0.440 to 0.443, and answer F1 from 0.573 to 0.576.

The cloud-scale reference results have the highest answer scores. Qwen 72B Corrective reaches answer accuracy 0.572, answer F1 0.701, supporting-title recall 0.877, and strict evidence recall 0.768. Llama 70B Corrective is the strongest overall HotpotQA condition, with answer accuracy 0.597, answer F1 0.732, supporting-title recall 0.885, and strict evidence recall 0.782.

Table 4.3: Overall HotpotQA results using the Section 3.7 metrics. Cloud-scale rows use the same retrieval and evaluation pipeline with larger matched models. Bold marks the strongest value in each reported quality column.

Condition	Answer Accuracy	Answer F1	Supporting-Title Recall	Strict Evidence Recall	Top- <i>k</i> Hit Rate	Retry Rate	Retrieval Calls Mean
<i>Local laptop-feasible runs</i>							
Qwen 3B Traditional	0.390	0.512	0.809	0.638	0.980	0.000	1.000
Qwen 3B Corrective	0.391	0.513	0.809	0.640	0.981	0.210	1.416
Qwen 7B Traditional	0.451	0.578	0.809	0.638	0.980	0.000	1.000
Qwen 7B Corrective	0.465	0.594	0.853	0.725	0.986	0.863	2.679
Llama 3B Traditional	0.364	0.508	0.809	0.638	0.980	0.000	1.000
Llama 3B Corrective	0.364	0.508	0.809	0.638	0.980	0.002	1.003
Llama 8B Traditional	0.440	0.573	0.809	0.638	0.980	0.000	1.000
Llama 8B Corrective	0.443	0.576	0.826	0.674	0.983	0.862	2.703
<i>Cloud-scale reference</i>							
Qwen 72B Traditional	0.528	0.658	0.809	0.638	0.980	0.000	1.000
Qwen 72B Corrective	0.572	0.701	0.877	0.768	0.988	0.345	1.669
Llama 70B Traditional	0.509	0.644	0.809	0.638	0.980	0.000	1.000
Llama 70B Corrective	0.597	0.732	0.885	0.782	0.993	0.476	1.944

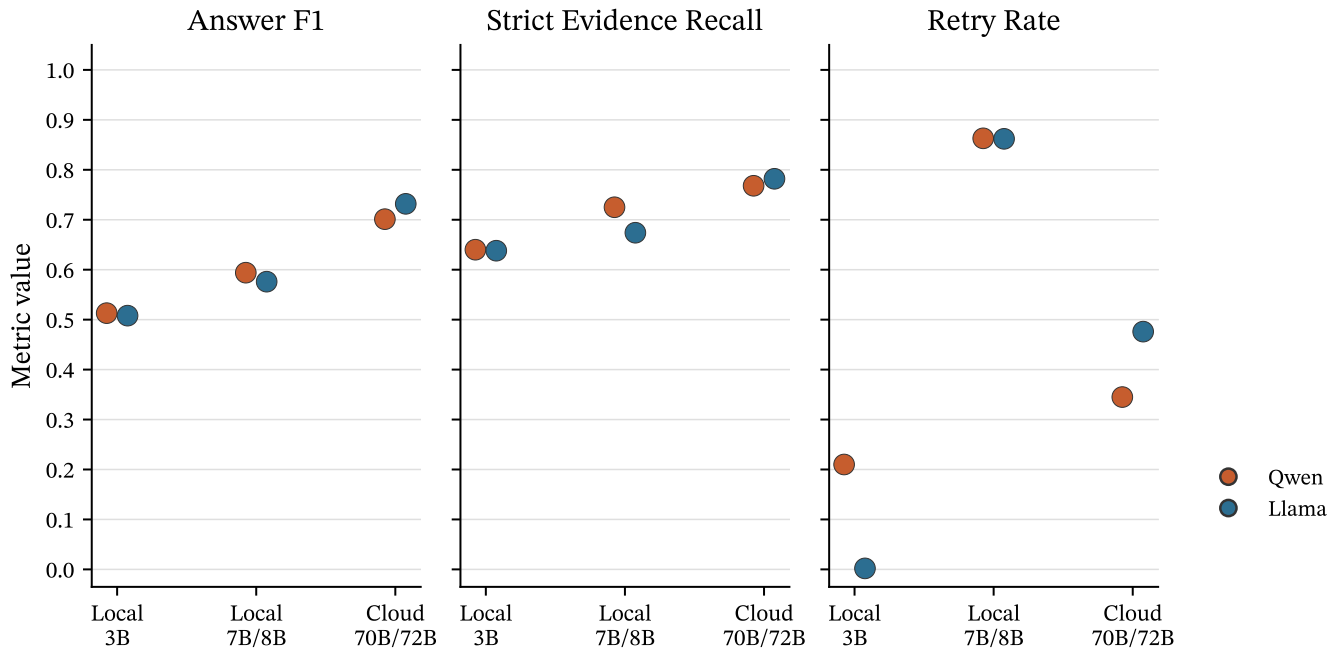


Figure 4.2: HotpotQA corrective pipeline results across local and cloud-scale model tiers.

4.3.2 Retrieval and Answer Effects of Correction

For local Qwen 7B, strict evidence recall rises by 0.087, from 0.638 to 0.725, while answer accuracy rises by 0.014, from 0.451 to 0.465, and answer F1 rises by 0.016, from 0.578 to 0.594. The paired uncertainty checks report strict evidence recall 95% CI $[+0.068, +0.107]$, $p < 0.001$; answer accuracy 95% CI $[+0.004, +0.025]$, $p = 0.013$; and answer F1 95% CI $[+0.006, +0.026]$.

For local Llama 8B, strict evidence recall rises by 0.036, from 0.638 to 0.674, while answer accuracy changes by 0.003, from 0.440 to 0.443, and answer F1 by 0.004, from

0.573 to 0.576. The paired uncertainty checks report strict evidence recall 95% CI $[+0.020, +0.053]$, $p < 0.001$; answer accuracy 95% CI $[-0.005, +0.011]$, $p = 0.607$; and answer F1 95% CI $[-0.005, +0.012]$.

At cloud scale, Qwen 72B Corrective improves over Qwen 72B Traditional by +0.044 answer accuracy, +0.043 answer F1, and +0.130 strict evidence recall. Llama 70B Corrective improves over Llama 70B Traditional by +0.088 answer accuracy, +0.088 answer F1, and +0.144 strict evidence recall.

Top-*k* hit rate is 0.980 in all HotpotQA traditional re-

sults and ranges from 0.980 to 0.993 under correction. Full paired uncertainty tables for the local comparisons are provided in Appendix E; binary metrics use exact two-sided McNemar tests, and mean-valued metrics use paired bootstrap 95% confidence intervals.

4.4 Retrieval-Width Diagnostics

Because the corrective pipeline changes more than retrieval width alone, additional diagnostics are needed to separate broader one-pass retrieval from the corrective loop itself. These diagnostics keep the generator fixed to local Qwen 7B, so the comparison is not affected by the cloud-scale model-size changes reported above.

4.4.1 LegalBench-mini

Table 4.4 summarizes the LegalBench-mini local Qwen 7B retrieval-width diagnostics.

Increasing the traditional one-pass retrieval width from $k = 5$ to $k = 20$ improves evidence recall from 0.308 to 0.384 and top- k hit rate from 0.308 to 0.465.

However, traditional $k = 20$ remains below the bounded corrective RAG. The main bounded corrective run reaches 0.519 evidence recall and 0.647 top- k hit rate. The remaining evidence-recall gap over traditional $k = 20$ is 0.135, corresponding to roughly 105 additional successful evidence-overlap cases out of 776. Moreover, reducing the corrective-stage retrieval width from $k_c = 20$ to $k_c = 5$ changes final evidence recall only slightly, from 0.519 to 0.514.

The top- k hit rate drops more clearly when k_c is reduced, from 0.647 to 0.548. Thus, the wider corrective stage improves candidate-set coverage, but much of that additional candidate coverage does not translate into final used-context evidence recall.

4.4.2 HotpotQA

Table 4.5 presents the HotpotQA local Qwen 7B retrieval-width diagnostic.

Table 4.4: LegalBench-mini Qwen 7B retrieval-width diagnostics. k_c denotes the corrective-stage retrieval width. Bold marks the strongest setting in this comparison.

Pipeline	Evidence Recall	Top- k Hit Rate
Traditional $k = 5$	0.308	0.308
Traditional $k = 20$	0.384	0.465
Bounded corrective $k_c = 5$	0.514	0.548
Bounded corrective $k_c = 20$	0.519	0.647

In this diagnostic, traditional $k = 20$ is slightly stronger than bounded corrective Qwen 7B across the reported answer and retrieval metrics. This HotpotQA failure set is dominated by partial rather than total misses. Under traditional $k = 5$, 638 of 1,000 cases recover both supporting pages, 342 recover exactly one, and only 20 recover neither. Bounded corrective Qwen 7B improves mainly by converting part of this large one-support group into full two-support recovery, reaching 725 cases with both supports recovered, but traditional $k = 20$ goes further still, reaching 747. Appendix E shows that the answer-accuracy and strict-recall differences between traditional $k = 20$ and bounded correction are not clearly decisive in the paired uncertainty checks, while supporting-title recall remains higher for the wider one-pass baseline.

A best-case answer-selection diagnostic estimates how much accuracy is lost at the final selection step. It is applied only to cases where the pipeline produced two candidate answers: a first-pass answer and a corrected-pass answer. The implemented selector chooses one of these two answers. The best-case selector is a hypothetical selector that always picks the correct candidate whenever either of the two answers is correct.

For Qwen 7B, correction activated and produced both candidate answers in 863 cases. On these cases, the implemented selector reached 45.0% exact-match accuracy. The best-case selector would reach 53.9%. Across the full 1,000-case run, the actual pipeline reached 46.5% exact-match accuracy, while the corresponding best-case upper bound is 54.2%. This indicates that the pipeline often generates a correct candidate answer, but the selector does not always choose it.

The same diagnostic for Llama 8B shows a different pattern. Correction activated and produced both candidate answers in 862 cases. On these cases, the implemented selector reached 41.6% exact-match accuracy, while the best-case selector would reach 49.7%. Across the full run, the actual pipeline reached 44.3% accuracy, compared with a best-case upper bound of 51.2%. However, in 434 of the 862 retried cases (50.3%), neither candidate answer is correct. In those cases, a better selector cannot

Table 4.5: HotpotQA Qwen 7B retrieval-width diagnostic. Bold marks the strongest setting in this comparison.

Pipeline	Answer Accuracy	Supporting-Title Recall	Strict Evidence Recall
Traditional $k = 5$	0.451	0.809	0.638
Traditional $k = 20$	0.473	0.868	0.747
corrective $k_c = 20$	0.465	0.853	0.725

help. This indicates both a selection-stage issue and an earlier pipeline issue, because many retried cases do not produce any correct answer to choose from.

4.5 Efficiency and Deployment Cost

The primary efficiency results in this section come from the controlled 100-query local latency study. Table 4.6 reports median latency, 95th-percentile latency, peak resident memory, and retrieval/generation latency breakdowns. The RTX pod reruns and H200 cloud-scale runs recorded per-query times, but they are not pooled into this section because they use different hardware and were not run under the same controlled local latency protocol. No controlled-latency condition recorded timeouts, errors, or stability warnings.

4.5.1 Latency and Memory

Table 4.6 shows that bounded corrective RAG is slower than traditional RAG in every model-benchmark pair. The largest median latencies occur in high-activation corrective conditions: 42.72 s for LegalBench-mini Qwen 7B, 43.68 s for HotpotQA Qwen 7B, and 52.49 s for HotpotQA Llama 8B. By contrast, traditional HotpotQA conditions remain below 5 s at P95, and traditional LegalBench-mini conditions remain below 17 s at P95.

Peak resident memory shows far less variation than latency. The 3B conditions require roughly 3.29 to 4.34 GiB, while the 7B and 8B conditions use approximately 5.63 to 6.32 GiB. Traditional and corrective pipelines differ little in memory usage within the same model. Thus, the primary efficiency cost of bounded correction is increased latency, rather than higher peak memory.

4.5.2 Controller Overhead and Sustained-Load Behavior

The clearest controller-overhead case is LegalBench-mini Llama 8B. As shown in Table 4.7, the traditional and corrective conditions have identical mean prompt and retrieved-context token loads (703 and 637, respectively),

yet median latency rises from 7.62 s to 16.78 s. In the high-activation conditions, token load also increases, especially for Qwen on both benchmarks and Llama 8B on HotpotQA, but the latency increase is larger than token growth alone. Low activation reduces the cost of the corrective pipeline relative to high-activation conditions, but it does not remove controller overhead.

HotpotQA Llama 3B shows similarly near-zero activation, with a retry rate of 0.002 and negligible quality change. A supplementary test for Qwen 3B and Qwen 7B using Apple `powermetrics` shows that dynamic RSS overhead above the loaded model remains small across conditions. Estimated energy per query broadly follows median latency; full results are reported in Appendix E (Tables E.4 and E.5).

Table 4.8 shows a second pattern: within the 10-query batches of the controlled-latency protocol, expensive corrective conditions exhibit clear sustained-load latency growth, whereas the traditional baselines remain comparatively flat. For example, HotpotQA Qwen 7B rises from 37.2 s at Position 1 to 47.0 s at Position 10, while the corresponding traditional condition changes only from 3.5 s to 3.6 s. Because the same batches were executed in both canonical and reversed query order, and the growth remains similar across both sweeps, this pattern is not explained by query difficulty alone.

Figure 4.3 summarizes this relationship by plotting the main quality metric for each benchmark against controlled median latency. The figure makes clear that the largest runtime increases are concentrated in a small number of highly corrective conditions, while the quality gains remain uneven across models and benchmarks.

Mean retrieval latency remains small across all conditions, ranging from 0.04 s to 0.10 s. In contrast, generation latency rises sharply in highly corrective runs, reaching 40.69 s for LegalBench-mini Qwen 7B, 43.14 s for HotpotQA Qwen 7B, and 49.10 s for HotpotQA Llama 8B. This indicates that the main runtime cost comes from additional model calls for control, rewriting, answer generation, and, for HotpotQA, answer selection.

The latency breakdown (see Appendix E, Figure E.1) shows that the added cost is reflected mainly in generation-side time rather than raw retrieval time.

4.5.3 Runtime Reference Across Deployment Settings

Table 4.9 lists median per-query latency from the benchmark runs across three deployment settings: laptop-local inference, an RTX pod using the same 7B/8B model size, and an H200 pod using the larger matched 70B/72B

Table 4.6: Controlled local latency summary on fixed 100-query manifests. Latency values pool two repeated sweeps; Peak RSS reports the maximum observed resident set size.

Panel A: LegalBench-mini							
Condition	Median (s)	P95 (s)	Peak RSS (GiB)	Retry Rate	Retrieval Calls Mean	Retrieval Mean (s)	Generation Mean (s)
Qwen 3B Traditional	3.70	5.23	4.11	0.00	1.00	0.04	3.77
Qwen 3B Corrective	17.43	22.92	4.34	0.95	2.90	0.09	16.83
Qwen 7B Traditional	6.39	9.54	5.63	0.00	1.00	0.05	6.58
Qwen 7B Corrective	42.72	58.29	5.96	0.93	2.86	0.10	40.69
Llama 3B Traditional	3.21	8.06	4.31	0.00	1.00	0.04	3.61
Llama 3B Corrective	6.11	11.26	4.32	0.00	1.00	0.04	6.45
Llama 8B Traditional	7.62	16.34	5.91	0.00	1.00	0.05	8.53
Llama 8B Corrective	16.78	29.08	6.29	0.00	1.00	0.05	17.75

Panel B: HotpotQA							
Condition	Median (s)	P95 (s)	Peak RSS (GiB)	Retry Rate	Retrieval Calls Mean	Retrieval Mean (s)	Generation Mean (s)
Qwen 3B Traditional	1.52	2.16	3.29	0.00	1.00	0.04	1.48
Qwen 3B Corrective	5.30	21.05	3.62	0.18	1.36	0.05	7.23
Qwen 7B Traditional	3.37	4.68	5.76	0.00	1.00	0.04	3.30
Qwen 7B Corrective	43.68	67.65	5.99	0.88	2.68	0.09	43.14
Llama 3B Traditional	1.54	2.07	3.53	0.00	1.00	0.04	1.49
Llama 3B Corrective	5.31	6.75	3.73	0.01	1.01	0.04	5.14
Llama 8B Traditional	3.46	4.75	6.27	0.00	1.00	0.04	3.41
Llama 8B Corrective	52.49	68.03	6.32	0.89	2.77	0.10	49.10

Table 4.7: Controller overhead and token pressure in the controlled-latency conditions. Prompt Tokens and Context Tokens are mean per-query token counts of the final prompt and the retrieved context inserted into it.

Condition	Prompt Tokens	Context Tokens	Median Latency (s)
Legal Qwen 7B Traditional	713	649	6.39
Legal Qwen 7B Corrective	1039	953	42.72
Legal Llama 8B Traditional	703	637	7.62
Legal Llama 8B Corrective	703	637	16.78
Hotpot Qwen 7B Traditional	614	558	3.37
Hotpot Qwen 7B Corrective	1145	1089	43.68
Hotpot Llama 8B Traditional	587	531	3.46
Hotpot Llama 8B Corrective	909	853	52.49

Table 4.8: Within-batch sustained-load latency growth in the controlled-latency study. Position 1 and Position 10 are pooled mean latencies in seconds across both sweep orderings. Similar growth under both orderings indicates that the pattern is not explained by query difficulty alone. Traditional Llama 8B rows are omitted for brevity (drift: 0.4 s on LegalBench-mini; 0.1 s on HotpotQA).

Condition	Pos. 1 (s)	Pos. 10 (s)	Drift (s)
Legal Qwen 7B Traditional	5.6	5.9	+0.3
Legal Qwen 7B Corrective	30.2	39.5	+9.3
Legal Llama 8B Corrective	13.6	18.9	+5.4
Hotpot Qwen 7B Traditional	3.5	3.6	+0.1
Hotpot Qwen 7B Corrective	37.2	47.0	+9.8
Hotpot Llama 8B Corrective	44.2	53.1	+8.9

model. These values are reported separately from Table 4.6 because they come from full benchmark runs rather than the repeated-sweep controlled local latency protocol.

The runtime reference gives the scale of the deployment effect. On the RTX pod, the same local model sizes run much faster than on the laptop full benchmark runs: Hot-

potQA Qwen 7B changes from 37.32 s to 1.69 s median latency, and HotpotQA Llama 8B changes from 43.08 s to 1.84 s. The H200 rows show that the larger 70B/72B models remain below 10 s median latency in these benchmark runs, while also changing the quality and controller-activation results reported earlier.

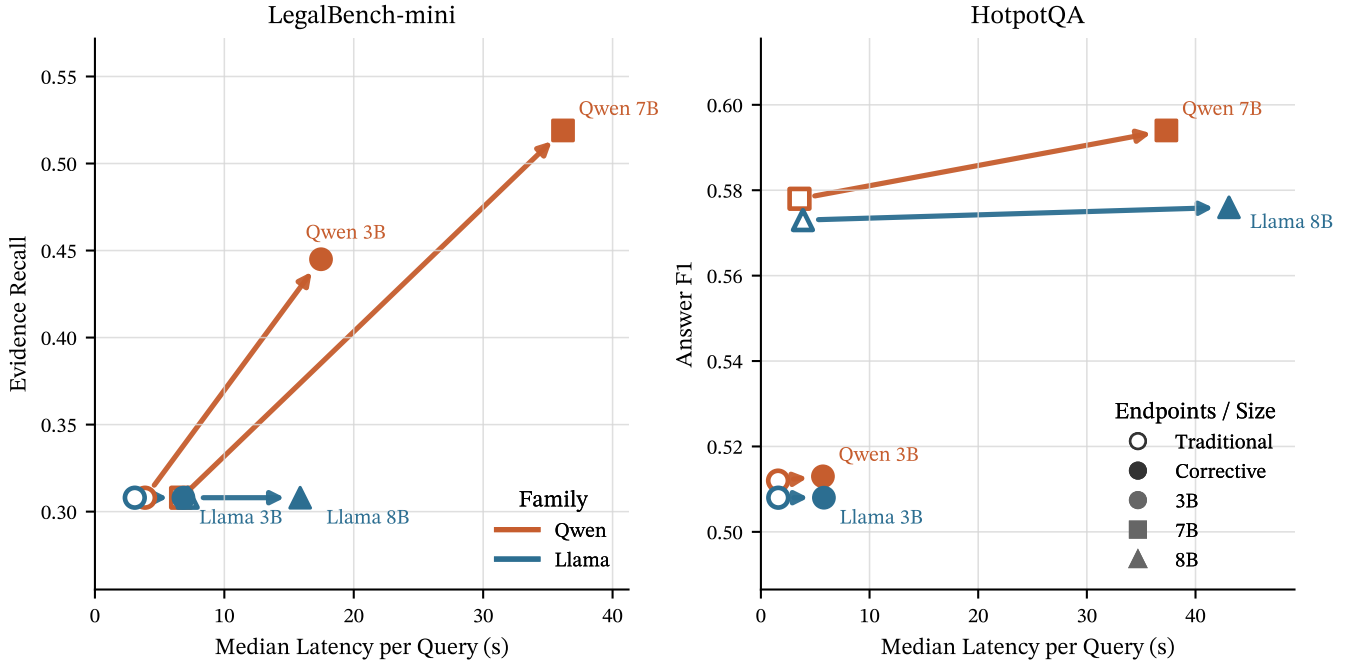


Figure 4.3: Quality–cost trade-off under controlled local latency measurement. Panel A plots LegalBench-mini evidence recall against controlled median latency; Panel B plots HotpotQA answer F1 against controlled median latency. Open markers denote traditional runs and filled markers denote corrective runs; arrows connect each traditional condition to its corrective counterpart. Quality values come from the full benchmark runs, whereas latency values come from the separate controlled 100-query latency study.

Table 4.9: Median per-query latency for corrective runs across deployment settings. Local and RTX rows use the same 7B/8B model size; H200 rows use the larger 70B/72B model from the same family.

Benchmark	Family	Local 7B/8B	RTX 7B/8B	H200 70B/72B
LegalBench-mini	Qwen	36.17	1.50	8.75
LegalBench-mini	Llama	15.86	0.84	7.14
HotpotQA	Qwen	37.32	1.69	3.26
HotpotQA	Llama	43.08	1.84	3.29

HotpotQA shows slightly more variation, but the differences remain small relative to the main results. Qwen 7B answer accuracy changes from 0.465 to 0.461 and supporting-title recall from 0.853 to 0.850. Llama 8B answer accuracy changes from 0.443 to 0.442 and supporting-title recall from 0.826 to 0.822. The full metric table is provided in Appendix E, Table E.6.

5 DISCUSSION

5.1 Corrective RAG as Retrieval Repair

The bounded corrective pipeline is best understood as a retrieval-repair mechanism. Its purpose is not to replace one-pass RAG, but to recover evidence that the first retrieval pass fails to place in the final context. The key question is whether correction still adds value when compared to a stronger one-pass baseline, not only with traditional $k = 5$.

On LegalBench-mini, the answer is mostly yes for Qwen 7B. Traditional $k = 5$ reaches gold evidence in 239 of 776 cases, while bounded correction reaches 403 cases. Increasing one-pass retrieval to $k = 20$ helps, reaching about 298 cases, but it still leaves roughly 105 evidence-recovery cases gained by the corrective pipeline. This suggests that the legal improvement is not caused only by retrieving more candidates. The correc-

4.6 Cross-Environment Validation

This section compares the final local runs with matched pod-based reruns. The goal is to test whether the main effectiveness and corrective-behavior trends remain stable across execution environments rather than reflecting one local machine. Latency and memory are excluded here because they are hardware-dependent and are analyzed separately in the efficiency section.

Figure 4.4 shows close agreement between local and pod reruns. On LegalBench-mini, the Llama traditional and corrective results remain unchanged, while the active Qwen corrective rows show only small differences. Qwen 3B evidence recall changes from 0.445 to 0.455, and Qwen 7B changes from 0.519 to 0.518. Retry rates remain unchanged for both Qwen LegalBench-mini runs.

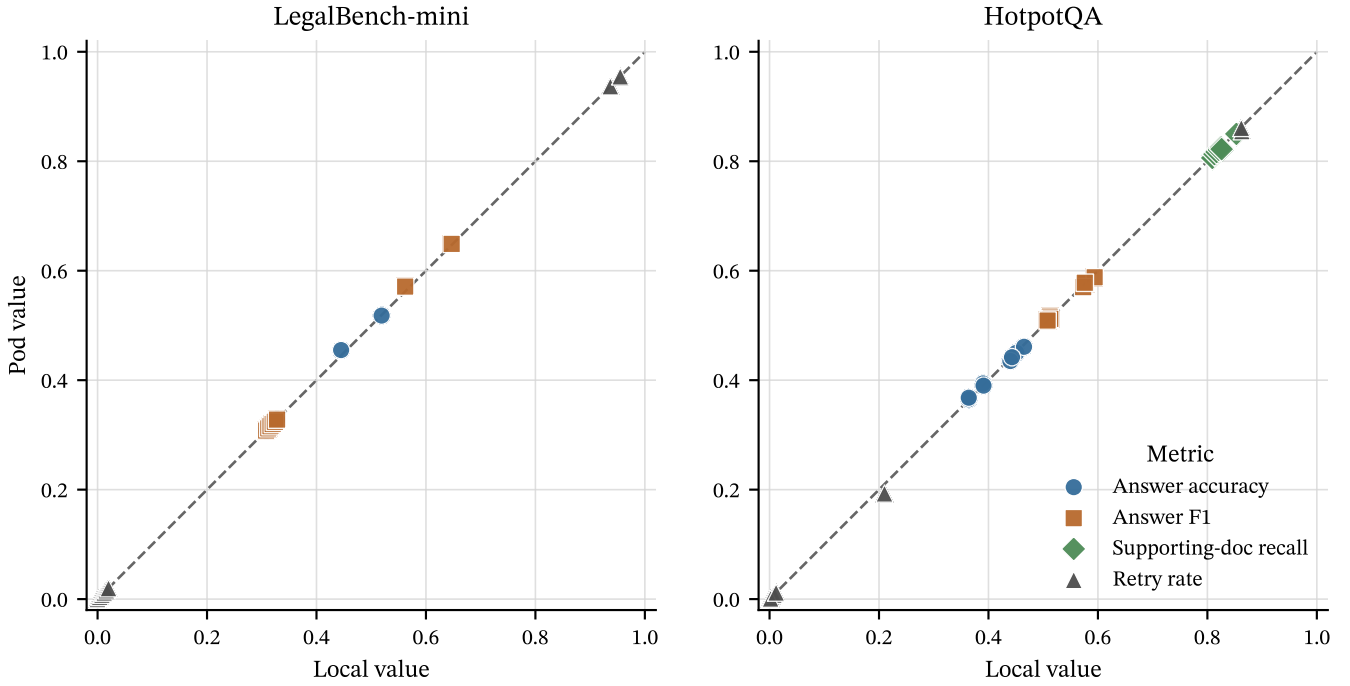


Figure 4.4: Agreement between local and pod-based matched runs on the metrics used in the cross-environment validation. Each point compares the local value on the horizontal axis with the corresponding pod value on the vertical axis; the dashed line indicates exact agreement. Most points lie close to the diagonal in both benchmarks, showing that the main effectiveness conclusions remain stable across execution environments even when small behavioural differences appear in a few bounded corrective conditions. The largest visible deviation is the HotpotQA Qwen 3B retry-rate point, but this behavioural difference does not materially alter the answer-level results.

tive stage changes the retrieval attempt through source-focused routing, query rewriting, backup retrieval, fusion, and final context construction.

The corrective-width diagnostic supports this interpretation. Reducing the corrective-stage width from $k_c = 20$ to $k_c = 5$ changes final evidence recall only slightly, from 0.519 to 0.514. If the gain mainly came from retrieving many more chunks, this reduction should have caused a larger drop. Instead, the legal benefit appears to come mainly from redirecting retrieval toward a better source region or clause-like match. For legal evidence review, this is practically meaningful because recovering the right supporting clause may justify a slower response time.

HotpotQA shows a different pattern within the closed-corpus setup used in this thesis. Qwen 7B correction raises strict evidence recall from 638 to 725 cases out of 1,000. However, traditional $k = 20$ reaches 747 cases, recovering 22 more cases than the corrective pipeline. Many HotpotQA failures are partial-coverage failures: under traditional $k = 5$, 342 questions already recover exactly one supporting title, while only 20 recover neither. In this setting, wider one-pass retrieval often brings the missing second title into the context without needing a controller or rewrite.

This does not mean that $k = 20$ is always preferable. Wider retrieval is attractive locally because it avoids extra controller, rewriting, and answer-selection calls. It is especially plausible in closed, well-indexed corpora such as internal wikis, documentation search, or help-center QA, where the original query is already close and the missing evidence is ranked nearby. However, increasing k cannot fix a poorly formulated query, a wrong source focus, or evidence that is far below the retrieved candidate set. It can also introduce irrelevant context, and under a fixed context budget, more retrieved candidates do not guarantee that the right evidence reaches the prompt.

The practical conclusion is task-dependent. For LegalBench-mini-like source-focused evidence recovery, bounded correction is more defensible because it recovers evidence that wider one-pass retrieval still misses. For the closed-corpus HotpotQA setup used here, correction is less clearly justified because wider one-pass retrieval gives comparable or stronger results with a simpler pipeline. This conclusion should not be generalized to the official open full-wiki HotpotQA setting, where retrieval from a much larger corpus may create different failure modes.

This task-dependent value makes the cost of correction part of the result, which is why the next section turns from

retrieval gains to deployment trade-offs.

5.2 Efficiency-Effectiveness Trade-offs Under Local Deployment

Local RAG is feasible, but the usable design depends on how much latency and energy the user is willing to spend for each additional recovered case. Local execution provides privacy, offline use, and direct corpus control, but every controller call, rewrite, and generation step becomes waiting time on the user’s device.

The local Qwen comparison shows the trade-off clearly. On LegalBench-mini, moving from Qwen 3B Corrective to Qwen 7B Corrective gives 58 additional evidence-recovered cases out of 776, while median controlled latency increases by 25.29s per query. This may be acceptable in legal evidence review. On HotpotQA, Qwen 7B Corrective gives 85 additional strict-evidence cases and 74 additional exact-match answers compared with Qwen 3B Corrective, but median latency increases by 38.38s. That is harder to justify for interactive question answering.

The local energy measurements reinforce this point. For Qwen 7B, corrective RAG increases estimated energy per query by 304.64 J on LegalBench-mini and 305.63 J on HotpotQA. This is about 4.3 times the traditional energy on LegalBench-mini and 6.3 times on HotpotQA. Peak RSS changes much less, by only 0.33 GiB and 0.23 GiB respectively. The laptop can run the pipeline, but the main cost of correction is repeated inference, not memory capacity.

The RTX pod results show what changes when the same 7B/8B models are moved off the laptop. Quality is not the main difference because the model size and pipeline are matched. The main difference is usability: corrective runs that take tens of seconds locally fall to around 1–2 seconds median latency on the RTX pod. This makes the same pipeline much more interactive, but it also removes the main advantage of laptop-local deployment because computation no longer stays fully on the user’s machine.

The H200 cloud-scale reference changes both hardware and model scale. On HotpotQA, the larger models provide meaningful gains. Qwen 72B Corrective adds 43 strict-evidence cases over local Qwen 7B Corrective, while Llama 70B Corrective adds 108 strict-evidence cases over local Llama 8B Corrective. Llama 70B also adds 154 exact-match answers over local Llama 8B and 132 over local Qwen 7B. In this closed-corpus HotpotQA setting, cloud-scale inference therefore provides a clear answer-quality gain.

LegalBench-mini is different. Qwen 72B Corrective

does not improve over local Qwen 7B Corrective, and Llama 70B Corrective remains below local Qwen 7B despite activating correction. This shows that larger models are not automatically better for retrieval-control tasks. For LegalBench-mini, the match between model behavior, controller decisions, and task structure matters more than parameter count alone.

The cloud-scale setting is therefore most defensible when the application values answer quality and can accept external GPU infrastructure. The H200 pod was priced at \$3.99/hour during the runs, and the Qwen 72B monitor log shows a median sampled GPU power of about 473 W. These values are not directly comparable to the controlled laptop energy measurements, but they indicate the scale of the deployment shift. For privacy-sensitive legal evidence recovery, the local Qwen 7B condition remains more compelling; for HotpotQA-like multi-document QA, the cloud-scale models provide stronger answer quality.

These trade-offs point back to the component that decides whether the extra cost is incurred in the first place: the controller.

5.3 The Controller Is a Central Bottleneck

A central limitation of the bounded corrective pipeline is that the same model is used for several roles: judging whether the first-pass context is sufficient, rewriting the query, generating the answer, and, for HotpotQA, selecting between first-pass and corrected-pass answers. This choice keeps the system fully local and avoids adding a separate benchmark-specific evaluator, but it also means that the controller is not trained specifically for evidence sufficiency. The pipeline therefore depends on how well each prompted model can decide when correction is needed.

The results suggest that controller behavior depends on both model family and model scale, rather than on hardware alone. On LegalBench-mini, the local Llama models never activate correction, both on the laptop and in the matched RTX rerun. However, Llama 70B activates correction on the same pipeline, with a retry rate of 0.661, and reaches 0.488 evidence recall. This indicates that the earlier Llama non-activation is not simply an implementation or hardware issue; the larger model follows the corrective instruction more effectively.

The Qwen models show a different pattern. Local Qwen 7B reaches the best LegalBench-mini result, but with very high activation: retry rate 0.955. This helps over-all evidence recovery, but it is not always selective. In contrast, Qwen 72B retries less often on LegalBench-mini but also performs below local Qwen 7B. Lower activation is

therefore not automatically better; the controller must retry the right cases.

HotpotQA gives a clearer example of selective correction at larger scale. Local Qwen 7B and Llama 8B retry on most queries, but the answer gains remain modest. The cloud-scale models retry less often, especially Qwen 72B at 0.345 and Llama 70B at 0.476, while producing the strongest answer and strict-evidence scores. In this setting, larger models appear to combine more selective correction with stronger answer generation.

A separate lightweight evaluator is a possible alternative, but it is not a guaranteed improvement. In CRAG, the retrieval evaluator is a separate fine-tuned model, while this thesis uses prompted sufficiency judgments from the same local model used for generation Yan et al. (2024). A dedicated evaluator would need to learn the right sufficiency criterion for each task: span-level evidence recovery in LegalBench-mini and multi-title support recovery in HotpotQA. If that evaluator is trained or calibrated on a different relevance signal, it may not transfer cleanly to these benchmarks.

The controller is therefore both a quality bottleneck and a latency bottleneck. If it retries too little, the corrective pipeline behaves like one-pass RAG. If it retries too often, it adds inference cost and may disturb useful first-pass evidence. Future work should test whether a dedicated sufficiency evaluator or routing module can improve this selectivity enough to justify its added complexity under local deployment constraints.

6 LIMITATIONS

We compare traditional one-pass RAG with the bounded corrective pipeline as complete systems. This supports a pipeline-level evaluation, but it limits what we can say about which internal step causes each gain or failure. The retrieval-width diagnostics help interpret the results, but we do not isolate every component of correction, such as source routing, query rewriting, backup retrieval, fusion, or answer selection.

The benchmark scope is limited. LegalBench-mini is retrieval-focused and does not provide gold short answers, hence evaluation of answer quality was not possible. HotpotQA is evaluated on a fixed 1,000-case slice in a local closed-corpus setup. The results should therefore be interpreted as within-study comparisons rather than official benchmark scores.

We keep the retrieval backbone fixed rather than exhaustively tuning it to find its best configurations. We use one dense embedding model, one FAISS setup, fixed chunk-

ing, fixed context settings, and fixed prompts so that the comparison focuses on what the correction step adds over the same one-pass pipeline.

Finally, our resource measurements have practical limits. We measure latency and process-level peak RSS consistently in the controlled local study, but RSS is not a full hardware memory breakdown. The power and energy values are derived from macOS `powermetrics`, so they support same-device comparison but should not be treated as instrument-grade measurements.

7 CONCLUSION AND FUTURE WORK

7.1 Conclusion

This thesis evaluated whether a bounded corrective RAG pipeline adapted for laptop-local inference provides enough value to justify its additional latency and computational cost compared with one-pass RAG baselines. The results show that the answer depends on the task and on the model used to control correction.

For LegalBench-mini, corrective RAG is useful with Qwen models. The best corrective run improves evidence recovery beyond both one-pass baselines, including the wider $k = 20$ baseline. This means that, in the legal setting, correction does more than retrieve extra text: it can recover evidence that the first query does not reach.

For HotpotQA, the result is different. In the closed-corpus setup used here, wider one-pass retrieval is often enough. The $k = 20$ diagnostic performs better than bounded correction for Qwen 7B, so the corrective loop is not clearly justified for this setting.

The model used as controller also matters. Qwen and Llama do not activate correction in the same way, and larger models do not automatically give better retrieval-control behavior. This is important because the corrective pipeline only helps when the controller retries useful cases and keeps useful evidence in the final context.

The local cost of correction comes mostly from extra model calls, not from retrieval. This makes correction more reasonable for slower evidence-focused work, such as legal document review, than for fast question answering where a stronger one-pass retriever may already be sufficient.

The cloud-scale runs add one final result: larger models improve HotpotQA answer quality, but they do not surpass the best local Qwen model on LegalBench-mini evidence recovery. The main conclusion is therefore that bounded corrective RAG should be used selectively. It is valuable when it recovers evidence that a strong one-pass pipeline

still misses, but it is not a default improvement over one-pass RAG.

7.2 Future Work

A clear next step is to study the corrective controller on its own. In this thesis, the same model was used for several tasks: judging context sufficiency, rewriting queries, generating answers, and, for HotpotQA, selecting between answers. Future work should compare this prompted controller to a smaller, task-specific sufficiency evaluator or routing module. It is important to measure both the quality of results and the local runtime cost in these comparisons.

Another direction is to compare bounded correction against stronger one-pass retrieval systems. The $k = 20$ diagnostic showed that retrieving more documents can explain some of the gains, especially for HotpotQA. Future work should test hybrid retrieval, reranking, alternative embeddings, and different ways of chunking or context-budget choices before attributing improvements to the correction method itself.

A third direction is to improve the use of retrieved evidence after correction. For HotpotQA, this means stronger answer selection or evidence-grounded generation, since better retrieval did not always produce better final answers. For legal tasks, future work should move beyond evidence recovery. It should evaluate whether the retrieved evidence and generated output are useful and faithful in a realistic legal-review workflow.

ACKNOWLEDGEMENTS

We are sincerely grateful to our promotor, Prof. Dr. Mihai Băce, for his guidance, constructive feedback, and patience throughout this project.

We would like to thank the professors and assistants of the faculty for their teaching and support during our studies. We are also thankful to the evaluation committee for their time and valuable feedback.

Finally, we thank our friends, classmates, and families for their support and encouragement.

BIBLIOGRAPHY

Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*.

Cormack, G. V., Clarke, C. L. A., and Buettcher, S.

(2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. ACM.

Council of Bars and Law Societies of Europe (2025). CCBE guidelines on the use of cloud computing by Bars and lawyers. Technical report, Council of Bars and Law Societies of Europe.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *The Eleventh International Conference on Learning Representations*.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., and Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Gerganov, G. et al. (2023). llama.cpp. GitHub.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Yang, A., et al. (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Guha, N., Nyarko, J., Ho, D. E., Ré, C., Chilton, A., Narayana, A., Chohlas-Wood, A., Peters, A., Waldon, B., Rockmore, D. N., Zambrano, D., Talisman, D., Hoque, E., Surani, F., Fagan, F., Sarfaty, G., Dickinson, G. M., Porat, H., Hegland, J., Wu, J., Nudell, J., Niklaus, J., Nay, J., Choi, J. H., Tobia, K., Hagan, M., Ma, M., Livermore, M., Rasumov-Rahe, N., Holzenberger, N., Kolt, N., Henderson, P., Rehaag, S., Goel, S., Gao, S., Williams, S., Gandhi, S., Zur, T., Iyer, V., and Li, Z. (2023). LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc.

Jeong, S., Baek, J., Cho, S., Hwang, S. J., and Park, J. C. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 7036–7050. Association for Computational Linguistics.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):248:1–248:38.

- Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., and Neubig, G. (2023). Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992. Association for Computational Linguistics.
- Johnson, J., Douze, M., and Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and tau Yih, W. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781. Association for Computational Linguistics.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. (2024). AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Ma, X., Gong, Y., He, P., Zhao, H., and Duan, N. (2023). Query rewriting for retrieval-augmented large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5303–5315. Association for Computational Linguistics.
- Meta AI (2024). Llama 3.2: Revolutionizing edge AI and vision. Meta AI Blog. Accessed: 2024.
- Min, S., Gururangan, S., Wallace, E., Shi, W., Hajishirzi, H., Smith, N. A., and Zettlemoyer, L. (2024). SILO language models: Isolating legal risk in a nonparametric datastore. In *The Twelfth International Conference on Learning Representations*. OpenReview.net.
- Park, T., Lee, G., and Kim, M.-S. (2025). MobileRAG: A fast, memory-efficient, and energy-efficient method for on-device RAG. *arXiv preprint arXiv:2507.01079*.
- Petroni, F., Piktus, A., Fan, A., Lewis, P., Yazdani, M., Cao, N. D., Thorne, J., Jernite, Y., Karpukhin, V., Maillard, J., Plachouras, V., Rocktäschel, T., and Riedel, S. (2021). KILT: a benchmark for knowledge intensive language tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2523–2544. Association for Computational Linguistics.
- Pipitone, N. and Alami, G. H. (2024). LegalBench-RAG: A benchmark for retrieval-augmented generation in the legal domain. *arXiv preprint arXiv:2408.10343*.
- Qwen Team (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Rau, D., Déjean, H., Chirkova, N., Formal, T., Wang, S., Clinchant, S., and Nikoulina, V. (2024). BERGEN: A benchmarking library for retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 7640–7663. Association for Computational Linguistics.
- Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992. Association for Computational Linguistics.
- Shuster, K., Poff, S., Chen, M., Kiela, D., and Weston, J. (2021). Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3784–3803. Association for Computational Linguistics.
- Tian, C., Qin, X., Tam, K., Li, L., Wang, Z., Zhao, Y., Zhang, M., and Xu, C. (2025). CLONE: Customizing LLMs for efficient latency-aware inference at the edge. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 563–585. USENIX Association.
- Trivedi, H., Balasubramanian, N., Khot, T., and Sabharwal, A. (2023). Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10014–10037. Association for Computational Linguistics.
- Wang, R., Gao, Z., Zhang, L., Yue, S., and Gao, Z. (2025). Empowering large language models to edge intelligence: A survey of edge efficient llms and techniques. *Computer Science Review*, 57:100755.

- Xu, P., Ping, W., Wu, X., McAfee, L., Zhu, C., Liu, Z., Subramanian, S., Bakhturina, E., Shoeybi, M., and Catanzaro, B. (2024). Retrieval meets long context large language models. In *The Twelfth International Conference on Learning Representations*.
- Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. (2024). Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics.
- Yuan, J., Li, H., Ding, X., Xie, W., Li, Y.-J., Zhao, W., Wan, K., Shi, J., Hu, X., and Liu, Z. (2025). Understanding and mitigating numerical sources of nondeterminism in LLM inference. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*.
- Zheng, H. S., Mishra, S., Chen, X., Cheng, H.-T., Chi, E. H., Le, Q. V., and Zhou, D. (2024). Take a step back: Evoking reasoning via abstraction in large language models. In *The Twelfth International Conference on Learning Representations*.

APPENDICES

A	Hardware and Runtime Configuration	A-1
	A.1 Hardware and Operating System	A-1
	A.2 Supplementary GPU Pod Configurations	A-1
	A.3 Python Environment and Core Libraries	A-1
	A.4 Local Generator Models	A-1
	A.5 Shared Inference and Retrieval Settings	A-1
B	Dataset and Benchmark Construction.	B-3
	B.1 LegalBench-mini Construction	B-3
	B.2 Balanced LegalBench-mini Subset for Controlled Studies	B-3
	B.3 HotpotQA Closed-Corpus Adaptation	B-3
C	Prompt Templates	C-4
	C.1 Shared Answer Prompt Template	C-4
	C.2 Final Answer System Prompts	C-4
	C.2.1 Legal Answer Prompt	C-4
	C.2.2 HotpotQA Answer Prompt	C-4
	C.3 Controller Decision Prompts	C-4
	C.3.1 Source-Focused Legal Sufficiency Check	C-4
	C.3.2 Global Multi-Document Sufficiency Check	C-4
	C.4 Corrective Query Planning Prompts	C-5
	C.4.1 Legal Rewrite Prompt	C-5
	C.4.2 Global Bridge-Planning Prompt	C-5
	C.5 Answer Selection Prompt	C-6
D	LegalBench-mini Subset Visualisation	D-7
E	Supplementary Diagnostics and Resource Profiles.	E-8
	E.1 Illustrative LegalBench-mini Query Traces	E-8
	E.2 Paired Uncertainty Checks	E-10
	E.3 Sentence-Level Rescoring for HotpotQA Qwen 7B	E-11
	E.4 Qwen 7B Supplementary Resource Profile.	E-11
	E.5 Qwen 3B Supplementary Resource Profile.	E-12

E.6 Latency Decomposition E-13

E.7 Cross-Environment Validation Details E-13

APPENDIX A : HARDWARE AND RUNTIME CONFIGURATION

This appendix documents the local machine and software stack used for the final local benchmark runs in this thesis. All final local evaluations were run on the same MacBook Air described below. Inference was run locally on-device through a Python and `llama.cpp` stack; no remote model APIs were used during evaluation.

A.1 Hardware and Operating System

- Machine: MacBook Air with Apple M4
- CPU: 10 cores
- GPU: integrated 10-core Apple GPU with Metal support
- Memory: 16 GB unified memory
- Operating system: macOS Tahoe 26.3.1 25D771280a (Darwin 25.3.0)

A.2 Supplementary GPU Pod Configurations

The main thesis evaluation is local. Two supplementary cloud configurations were used only for robustness checks and deployment-reference runs:

- Matched rerun pod: RunPod on-demand instance with one NVIDIA GeForce RTX 4090 GPU, 24 GB VRAM, 16 vCPUs, and 62 GB RAM. The listed on-demand price during the runs was \$0.69/hour.
- Cloud-scale reference pod: RunPod on-demand H200 SXM instance with one NVIDIA H200 GPU, 141 GB VRAM, 24 vCPUs, and 377 GB RAM. The listed on-demand price during the runs was \$3.99/hour. The pod used the RunPod PyTorch 2.4.0 image with Python 3.11, CUDA 12.4.1, and Ubuntu 22.04; `nvidia-smi` reported driver 565.57.01 and CUDA 12.7.

The RTX pod was used for matched environment reruns with the same local model sizes. The H200 pod was used for the Qwen 2.5 72B and Llama 3.1 70B cloud-scale reference runs. Timing values from these pods are treated as deployment runtime references, not as part of the controlled local latency protocol.

A.3 Python Environment and Core Libraries

The repository uses a local virtual environment (`.venv`) with Python 3.12.12. The core library versions verified

from that environment are:

- `llama-cpp-python` 0.2.90
- `sentence-transformers` 2.7.0
- `faiss-cpu` 1.9.0.post1
- `transformers` 4.40.0
- `torch` 2.10.0
- `pandas` 2.2.0
- `numpy` 1.26.0

A.4 Local Generator Models

The evaluated local generator models were loaded from GGUF quantized files:

- `Llama-3.2-3B-Instruct-Q4_K_M.gguf`
- `Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf`
- `qwen2.5-3b-instruct-q4_k_m.gguf`
- `qwen2.5-7b-instruct-q4_k_m-00001-of-00002.gguf` and `qwen2.5-7b-instruct-q4_k_m-00002-of-00002.gguf`

A.5 Shared Inference and Retrieval Settings

Table A.1 summarizes the fixed runtime limits used in the main local evaluation and the reasons for using them. These values were not tuned to the smallest possible limits; instead, they were kept fixed as task-matched runtime settings with headroom for the evaluated runs.

The remaining shared retrieval configuration used by the main pipelines is:

- GPU offload setting: `n_gpu_layers = -1`
- embedding model: `BAAI/bge-base-en-v1.5`
- embedding normalization: enabled
- vector index: FAISS
- chunk target size: 800 characters
- chunk overlap: 150 characters
- default dense retrieval width: `k = 5`
- corrective retrieval width: `k_c = 20`
- maximum total retrieval iterations in the corrective loop: 2 (initial pass plus at most one corrective retry)
- reranking: disabled in the main local configuration

Table A.1: Fixed runtime and context settings used in the main local evaluation.

Setting	Value	Used for	Reason
Runtime context limit	4096 tokens	All runs	Fixed local runtime limit shared across all models. In the saved final runs, prompt length remained well below this limit, with a maximum of 1872 tokens.
Retrieved-context budget	6000 characters	Context assembly before generation	Context assembly operates on raw text before model-specific tokenization, so a character budget gives one common limit across all model conditions. This value is also above the baseline case of five 800-character chunks.
Seed	42	All runs	Kept fixed for reproducibility across repeated runs.
Temperature	0.0	All runs	Kept fixed to reduce output variation in the controlled comparison.
Generation limit for legal answers and longer intermediate outputs	512 tokens	LegalBench-mini answers and longer intermediate generations	Used as one common ceiling for longer outputs. In the saved LegalBench runs, 99% of generated answers remained below 328 tokens.
Generation limit for HotpotQA final answers	32 tokens	HotpotQA final answer generation	HotpotQA uses short answers. In the local validation set, 95% of gold answers contain five words or fewer.
Generation limit for controller decisions and answer selection	48 tokens	Sufficiency decisions and HotpotQA answer selection	These prompts require only very short outputs.
Generation limit for query rewriting and HotpotQA refinement planning	128 tokens	Query rewriting and refinement planning	These prompts return short structured query plans rather than full answers.

APPENDIX B : DATASET AND BENCHMARK CONSTRUCTION

This appendix documents how the benchmark inputs used in the thesis were prepared and which fixed subsets were used for the main local comparisons and shorter controlled studies.

B.1 LegalBench-mini Construction

The main legal benchmark reported in the thesis is `legalbenchrag_mini.json`. This file was derived locally from the official LegalBench-RAG mini sampling logic by `scripts/create_legalbench_mini.py`. Its metadata records a cap of 194 tests per source benchmark, with cases sorted by document before truncation, for a total of 776 test cases.

The resulting LegalBench-mini benchmark contains four equally sized source subsets:

- `privacy_qa`: 194 cases

- `contractnli`: 194 cases

- `maud`: 194 cases

- `cuad`: 194 cases

Each test row stores a natural-language query and one or more annotated evidence snippets. Each snippet annotation includes a source document path and a character span, which are used by the evaluation code for evidence-overlap scoring. The corresponding local legal corpus is organized under `data/corpus` in the same four subset directories.

This construction follows the published LegalBench-RAG-mini setup, which selects 194 queries from each source dataset for a lightweight benchmark configuration Pipitone and Alami (2024). We retained this setup for reproducibility and comparability with the benchmark as introduced by the original authors. The legal results in this thesis should therefore be interpreted as applying to this fixed LegalBench-RAG-mini setting rather than as a uniform re-sampling of the full LegalBench-RAG distribution.

B.2 Balanced LegalBench-mini Subset for Controlled Studies

For the controlled-latency study and the supplementary resource-profiling study, the thesis uses a separate balanced legal subset file, `legalbenchrag_mini_balanced.json`. This file was created from `legalbenchrag_mini.json` by `scripts/create_legalbench_mini_balanced.py` with seed 42. It contains 25 cases from each of the four legal subsets, for a total of 100 cases.

B.3 HotpotQA Closed-Corpus Adaptation

The HotpotQA data used in the thesis was prepared locally by `scripts/prepare_hotpotqa.py` from the Hugging Face `hotpot_qa` dataset, using the `distractor` configuration and the `validation` split. This preparation step produced two local JSONL files:

- `hotpotqa_validation_cases.jsonl`: 7,405 question-answer cases
- `hotpotqa_validation_corpus.jsonl`: 66,581 article-title documents

Each case stores a question, a gold short answer, and a list of supporting page titles. In the prepared validation file, every case has exactly 2 supporting titles. The local corpus file stores one text record per article title, assembled from the sentence lists provided in the validation split context.

APPENDIX C : PROMPT TEMPLATES

This appendix records the prompt templates used by the final evaluated local pipelines. Only prompts used in the final thesis configuration are reproduced below. The prompts are defined in `src/rag/prompts.py`. Placeholders such as `{context}`, `{question}`, and `{entity_hints}` are filled at runtime.

C.1 Shared Answer Prompt Template

The following user prompt template is shared across the traditional and corrective answer-generation paths:

```
Context:
{context}

Question:
{question}

Answer:
```

C.2 Final Answer System Prompts

C.2.1 Legal Answer Prompt

```
You are a careful legal assistant.
Answer the question using only the
provided context. If the context is
insufficient, say you don't know.
```

C.2.2 HotpotQA Answer Prompt

```
Answer the question using only the
provided context. Return only the final
short answer text. Do not add
explanation, prefix, or extra words.
```

C.3 Controller Decision Prompts

C.3.1 Source-Focused Legal Sufficiency Check

```
You are a retrieval evaluator for a
local RAG system. Judge only whether
the current retrieval is good enough to
answer the question. Be strict about
retrieval quality, clause specificity,
and source consistency, but do not
request another search if the needed
evidence is already present. If the
question refers to a specific agreement,
party, or document, treat wrong-source
retrieval as insufficient. If the
chunks are from the right document and
already contain the likely answer span,
```

```
clause heading, date, defined term,
obligation, exception, covenant, or
location cue needed to answer, return
STOP. Do not ask for a retry just
because a cleaner or more precise chunk
might exist. Use RETRY only when the
current chunks are wrong-source,
mixed-source in a harmful way, or
clearly missing the specific fact,
clause, condition, exception, covenant,
term, or answer span needed. Return
only STOP or RETRY.
```

```
Question:
{question}
```

```
Retrieved Context:
{context}
```

```
Decide whether the retrieved context is
sufficient to answer the question.
Return STOP if the retrieval is
relevant, source-consistent, and already
contains enough evidence to answer. If
the likely answer span or clause is
already present, return STOP even if
another search might find a cleaner
snippet. Otherwise return RETRY. Return
only one word.
```

C.3.2 Global Multi-Document Sufficiency Check

```
You are a retrieval evaluator for a
local RAG system handling open-domain or
multi-document questions. Judge only
whether the current retrieval already
contains enough evidence to answer the
question. Be strict about evidence
completeness, but do not request another
search if the current chunks already
provide the needed entities, bridge
facts, or comparisons. Return STOP when
the retrieved context already supports
the answer with enough connected
evidence, even if a cleaner second
search might exist. Return RETRY only
when the retrieval looks partial,
one-sided, or likely misses a needed
bridge fact, comparison target, or
second source. Return only STOP or
RETRY.
```

```
Question:
{question}
```

```
Retrieved Context:
{context}
```

```
Decide whether the retrieved context
already contains enough connected
evidence to answer the question. Return
```

STOP if the current context already includes the needed entities, bridge facts, comparisons, or supporting sources. Return RETRY only if the evidence still looks partial, one-sided, or missing a likely second source or bridge fact. Return only one word.

C.4 Corrective Query Planning Prompts

C.4.1 Legal Rewrite Prompt

You are a legal retrieval strategist for a local RAG system. Your job is to improve retrieval when the current chunks are insufficient. First identify the single most important fact missing from the retrieved context that prevents answering the question. Then write one targeted retrieval query for that missing fact. If useful, you may provide one backup query, but only if it targets the same missing fact from a different legal phrasing. Preserve important entities from the original question, including agreement names, party names, jurisdictions, dates, and distinctive terms. Preserve polarity and negation. Use compact retrieval-friendly legal wording such as likely clause terms, obligations, exceptions, conditions, and scope terms. Prefer short keyword-style search queries, not full questions or sentences. Keep each query under 18 words. Do not invent facts, section numbers, or entities not present in the question. Do not broaden to generic legal topics. Return JSON only.

Original Question:
{question}

Entity Hints To Preserve:
{entity_hints}

Retrieved Context:
{context}

Identify the most important missing fact needed to answer the question. Write compact keyword-style retrieval queries, not natural-language questions.

Return JSON with this schema:

```
{
  "missing_fact": "<short phrase>",
  "primary_query": "<short keyword query>",
  "backup_query": "<optional short keyword query or NONE>"
}
```

```
}
```

C.4.2 Global Bridge-Planning Prompt

You are a retrieval strategist for a local RAG system handling open-domain or multi-document questions. Your job is to decide whether a second global retrieval is actually needed. If the current retrieval already contains enough connected evidence to answer, do not request another search. If another retrieval is needed, step back from the original wording and identify the more general missing relation, role, location, office, time period, comparison attribute, or bridge concept that would connect the evidence. Write one primary step-back query for that missing abstract relation or bridge concept. If useful, write one secondary query for a concrete entity-focused follow-up that complements the first query rather than paraphrasing it. Preserve important entities, polarity, and comparison structure from the original question. Prefer compact keyword-style retrieval queries, not full sentences. Keep each query under 18 words. Do not invent unsupported facts or entities. Return JSON only.

Original Question:
{question}

Entity Hints To Preserve:
{entity_hints}

Retrieved Context:
{context}

Decide whether another global retrieval is needed. If not needed, set need_retry to NO and set both queries to NONE. If needed, set need_retry to YES and write a step-back query for the missing relation or bridge concept. Use backup_query only when it adds a complementary concrete follow-up, not just a rephrasing.

Return JSON with this schema:

```
{
  "need_retry": "<YES or NO>",
  "missing_fact": "<short phrase describing the missing evidence>",
  "primary_query": "<short keyword step-back query or NONE>",
  "backup_query": "<optional short complementary follow-up query or NONE>"
}
```

C.5 Answer Selection Prompt

The HotpotQA corrective pipeline uses an additional answer-selection step when both first-pass and corrected-pass answers are available.

You are a careful answer selector for open-domain question answering. You will receive a question, candidate answer A with its supporting context, and candidate answer B with its supporting context. Choose the candidate that is more directly and specifically supported by its context. Prefer exact supported answers over vague, partial, or hallucinated ones. If one candidate is unknown and the other is directly supported, choose the supported one. If neither candidate is directly supported, choose unknown. Return JSON only.

Question:

{question}

Candidate A Answer:

{answer_a}

Candidate A Context:

{context_a}

Candidate B Answer:

{answer_b}

Candidate B Context:

{context_b}

Return JSON with this schema:

```
{
  "choice": "<A or B or UNKNOWN>",
  "reason": "<short phrase>"
}
```

APPENDIX D : LEGALBENCH-MINI SUBSET VISUALISATION

This appendix contains the supplementary LegalBench-mini subset heatmap. The figure makes the subset pattern visible, especially the contrast between the broad Qwen gains and the `contractnli` decline under corrective Qwen 7B.

Table 4.2 in the main text reports the subset evidence-recall values numerically. The following heatmap presents the same values visually.

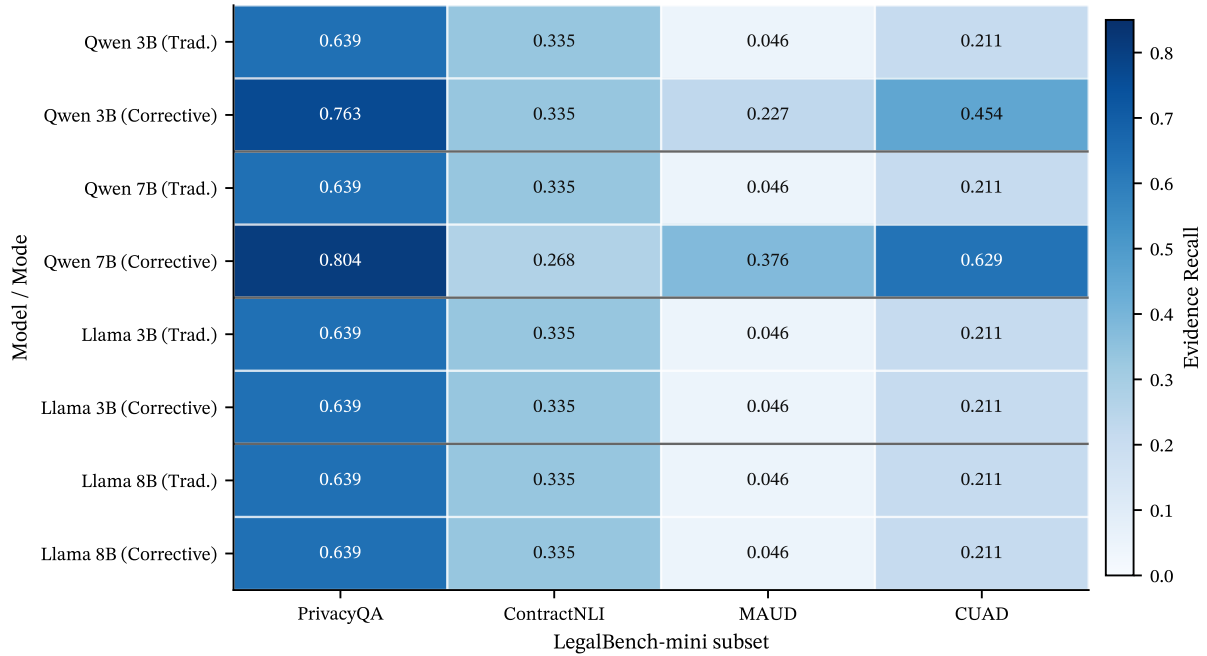


Figure D.1: LegalBench-mini subset evidence-recall heatmap on the final local runs. Cell values reproduce the per-subset evidence-recall values reported in Table 4.2.

APPENDIX E : SUPPLEMENTARY DIAGNOSTICS AND RESOURCE PROFILES

This appendix presents supplementary analyses that extend the main results without repeating the core tables from Chapter 4. It includes:

- a sentence-level rescoring check for the main HotpotQA Qwen 7B comparison;
- dedicated local resource-profiling tables reporting estimated power, estimated energy per query, and dynamic memory overhead beyond the latency and peak-memory summaries already reported in the main text.

The power and energy values are derived from macOS `powermetrics`, so they are useful for comparing runs on the same laptop but should not be interpreted as exact watt or joule measurements.

E.1 Illustrative LegalBench-mini Query Traces

The following four cases illustrate LegalBench-mini corrective behavior for Qwen 7B. They are presented in a compact trace format showing the query, retrieval state, controller decision, rewrite, route, and outcome.

Case 1: Successful source correction Subset: PrivacyQA

Outcome: Recovery

Query: Consider "Wordscapes"'s privacy policy; does it read my contacts?

First pass: Gold evidence not in used context; top chunk from `23andMe.txt`.

Controller: `RETRY`

Route: `doc_narrow` to `Wordscapes.txt`

Primary query: `read contacts privacy policy Wordscapes`

Backup query: `access contacts privacy policy Wordscapes`

Final pass: Gold evidence recovered; top chunk from `Wordscapes.txt`.

Why this happened: The first global retrieval was pulled toward generic contact-information language from other privacy policies, while the corrective step narrowed retrieval to `Wordscapes.txt` and recovered the gold contacts clause.

Case 2: Successful within-document recovery Subset: ContractNLI

Outcome: Recovery

Query: Consider the Non-Disclosure Agreement between CEII and NDA; Does the document include a clause that prevents the Receiving Party from disclosing the fact that the Agreement was agreed upon or negotiated?

First pass: Gold evidence not in used context; top chunk still from `ceii-and-nda.txt`.

Controller: `RETRY`

Route: `doc_narrow` to `ceii-and-nda.txt`

Primary query: `non-disclosure negotiations parties`

Backup query: `disclose fact agreement negotiations`

Final pass: Gold evidence recovered; top chunk remained in `ceii-and-nda.txt`.

Why this happened: The first pass stayed in the correct agreement but missed the relevant clause span, and the corrective query recovered it by searching again within `ceii-and-nda.txt`.

Case 3: Harmful retry through clause drift Subset: ContractNLI

Outcome: Regression

Query: Consider Motorola's Non-Disclosure Agreement; Does the document indicate that the Agreement does not grant the Receiving Party any rights to the Confidential Information?

First pass: Gold evidence already present in used context; top chunk from `nda_form_motorola.txt`.

Controller: RETRY

Route: `doc_narrow` to `nda_form_motorola.txt`

Primary query: Agreement grant Receiving Party rights Confidential Information

Backup query: Agreement grant Receiving Party use Confidential Information

Final pass: Gold evidence lost; top chunk shifted to `NCDG_Non-disclosure-agreement.txt`.

Why this happened: The first pass already contained the gold Motorola clause, but the rights-focused rewrite shifted the second pass toward a different non-disclosure agreement and displaced the correct evidence.

Case 4: Harmful retry through wrong-document routing Subset: ContractNLI

Outcome: Regression

Query: Consider HNBA's Confidentiality Agreement; Does the document require the Receiving Party to notify the Disclosing Party if they are required by law, regulation, or judicial process to disclose any Confidential Information?

First pass: Gold evidence already present in used context; top chunk from `HNBA-2017-18-Confidentiality-Agreement.txt`.

Controller: RETRY

Route: `doc_narrow` to `ENERGOUSCORP_03_16_2017-EX-10.24-STRATEGIC ALLIANCE AGREEMENT.txt`

Primary query: notice requirement legal disclosure confidential information

Backup query: write notice within five days legal requirement

Final pass: Gold evidence lost; top chunk shifted to `IPTK-CO-MutualNon-DisclosureAgreement.txt`.

Why this happened: The first pass already contained the gold HNBA clause, but the corrective route narrowed to the wrong agreement and removed the correct HNBA evidence from the final context.

E.2 Paired Uncertainty Checks

The following tables report paired uncertainty checks for the main local comparisons. Binary metrics use exact two-sided McNemar tests on paired case outcomes together with paired bootstrap percentile confidence intervals for the mean delta. Mean-valued HotpotQA metrics use paired bootstrap percentile confidence intervals only. All bootstrap intervals use 20,000 resamples with seed 42.

Table E.1: Paired uncertainty checks for binary metrics. Deltas are compare minus baseline. Confidence intervals are paired bootstrap percentile intervals (20,000 resamples, seed 42), and p -values are exact two-sided McNemar tests on paired case outcomes.

Panel A: Main traditional-versus-corrective comparisons							
Condition	Metric	Base	Compare	Δ	95% CI	Wins/Losses	p
LegalBench-mini Qwen 3B	Evidence recall	0.308	0.445	+0.137	[+0.106, +0.168]	134/28	≤ 0.001
LegalBench-mini Qwen 7B	Evidence recall	0.308	0.519	+0.211	[+0.175, +0.247]	201/37	≤ 0.001
LegalBench-mini Llama 3B	Evidence recall	0.308	0.308	+0.000	[+0.000, +0.000]	0/0	1.000
LegalBench-mini Llama 8B	Evidence recall	0.308	0.308	+0.000	[+0.000, +0.000]	0/0	1.000
HotpotQA Qwen 3B	Answer accuracy	0.390	0.391	+0.001	[-0.006, +0.008]	7/6	1.000
HotpotQA Qwen 3B	Strict recall	0.638	0.640	+0.002	[-0.008, +0.012]	15/13	0.851
HotpotQA Qwen 7B	Answer accuracy	0.451	0.465	+0.014	[+0.004, +0.025]	21/7	0.013
HotpotQA Qwen 7B	Strict recall	0.638	0.725	+0.087	[+0.068, +0.107]	99/12	≤ 0.001
HotpotQA Llama 3B	Answer accuracy	0.364	0.364	+0.000	[+0.000, +0.000]	0/0	1.000
HotpotQA Llama 3B	Strict recall	0.638	0.638	+0.000	[+0.000, +0.000]	0/0	1.000
HotpotQA Llama 8B	Answer accuracy	0.440	0.443	+0.003	[-0.005, +0.011]	9/6	0.607
HotpotQA Llama 8B	Strict recall	0.638	0.674	+0.036	[+0.020, +0.053]	54/18	≤ 0.001
Panel B: HotpotQA Qwen 7B width diagnostic							
Condition	Metric	Base	Compare	Δ	95% CI	Wins/Losses	p
HotpotQA Qwen 7B width	Answer accuracy	0.473	0.465	-0.008	[-0.029, +0.013]	52/60	0.509
HotpotQA Qwen 7B width	Strict recall	0.747	0.725	-0.022	[-0.045, +0.001]	57/79	0.071

Table E.2: Paired bootstrap uncertainty intervals for mean-valued HotpotQA metrics. Deltas are compare minus baseline, using 20,000 paired bootstrap resamples with seed 42.

Panel A: Main traditional-versus-corrective comparisons						
Condition	Metric	Base	Compare	Δ	95% CI	
HotpotQA Qwen 3B	Answer F1	0.512	0.513	+0.001	[-0.006, +0.008]	
HotpotQA Qwen 3B	Supp. recall	0.809	0.809	+0.001	[-0.005, +0.006]	
HotpotQA Qwen 7B	Answer F1	0.578	0.594	+0.016	[+0.006, +0.026]	
HotpotQA Qwen 7B	Supp. recall	0.809	0.853	+0.044	[+0.034, +0.054]	
HotpotQA Llama 3B	Answer F1	0.508	0.508	-0.000	[-0.000, +0.000]	
HotpotQA Llama 3B	Supp. recall	0.809	0.809	+0.000	[+0.000, +0.000]	
HotpotQA Llama 8B	Answer F1	0.573	0.576	+0.004	[-0.005, +0.012]	
HotpotQA Llama 8B	Supp. recall	0.809	0.826	+0.017	[+0.008, +0.025]	
Panel B: HotpotQA Qwen 7B width diagnostic						
Condition	Metric	Base	Compare	Δ	95% CI	
HotpotQA Qwen 7B width	Answer F1	0.605	0.594	-0.011	[-0.030, +0.009]	
HotpotQA Qwen 7B width	Supp. recall	0.868	0.853	-0.015	[-0.026, -0.003]	

E.3 Sentence-Level Rescoring for HotpotQA Qwen 7B

The following table reports a sentence-level rescoring pass for the main HotpotQA Qwen 7B comparison against original HotpotQA supporting-fact sentences. It complements the document-level retrieval metrics reported in the main results chapter.

Table E.3: Supplementary sentence-level rescoring for the main HotpotQA Qwen 7B comparison. Document-level values reproduce the reported local metrics exactly; sentence-level values rescore the same 1,000-case slice against original HotpotQA supporting-fact sentences.

Pipeline	Acc.	F1	Doc. supp.	Sent. supp.	Doc. strict	Sent. strict
Traditional $k = 5$	0.451	0.578	0.809	0.807	0.638	0.631
Traditional $k = 20$	0.473	0.605	0.868	0.865	0.747	0.741
Bounded corrective $k_c = 20$	0.465	0.594	0.853	0.851	0.725	0.716

E.4 Qwen 7B Supplementary Resource Profile

The Qwen 7B resource profile adds system-level estimated power and energy measurements, together with dynamic RSS overhead above the loaded-model baseline, for the four Qwen 7B benchmark-mode conditions.

Table E.4: Supplementary controlled resource profile for the Qwen 7B local runs. The main text already reports latency, retry behaviour, retrieval calls, and peak resident memory for these conditions; this appendix therefore adds only the new resource measures from the dedicated profiling runs. Values are means across the two repeated 20-query runs. For estimated average power, estimated energy per query, and dynamic RSS overhead, the absolute repeat range is reported in parentheses. Dynamic RSS overhead is computed as peak RSS minus idle loaded-model RSS.

Benchmark	Mode	Est. Power (W)	Est. Energy / Query (J)	Dyn. RSS Overhead (GB)
LegalBench-mini	Traditional	14.82 (0.236)	92.15 (0.71)	0.285 (0.0108)
LegalBench-mini	Corrective	10.34 (0.004)	396.79 (0.25)	0.532 (0.0260)
HotpotQA	Traditional	17.40 (0.070)	57.46 (0.06)	0.054 (0.0003)
HotpotQA	Corrective	10.00 (0.498)	363.09 (10.72)	0.571 (0.0007)

E.5 Qwen 3B Supplementary Resource Profile

The matching Qwen 3B table provides the same supplementary resource measures for the smaller Qwen model, allowing a same-family scale comparison under the local runtime stack.

Table E.5: Supplementary controlled resource profile for the Qwen 3B local runs. The main text already reports latency, retry behaviour, retrieval calls, and peak resident memory for these conditions; this appendix therefore adds only the new resource measures from the dedicated profiling runs. Values are means across the two repeated 20-query runs. For estimated average power, estimated energy per query, and dynamic RSS overhead, the absolute repeat range is reported in parentheses. Dynamic RSS overhead is computed as peak RSS minus idle loaded-model RSS.

Benchmark	Mode	Est. Power (W)	Est. Energy / Query (J)	Dyn. RSS Overhead (GB)
LegalBench-mini	Traditional	14.33 (0.046)	55.13 (0.08)	0.361 (0.0006)
LegalBench-mini	Corrective	12.02 (0.024)	215.78 (0.005)	0.633 (0.00005)
HotpotQA	Traditional	16.47 (0.138)	25.88 (0.12)	0.083 (0.0003)
HotpotQA	Corrective	14.19 (0.338)	119.98 (1.68)	0.108 (0.0001)

E.6 Latency Decomposition

The following figure separates retrieval-side and generation-side mean latency for all controlled-latency conditions.

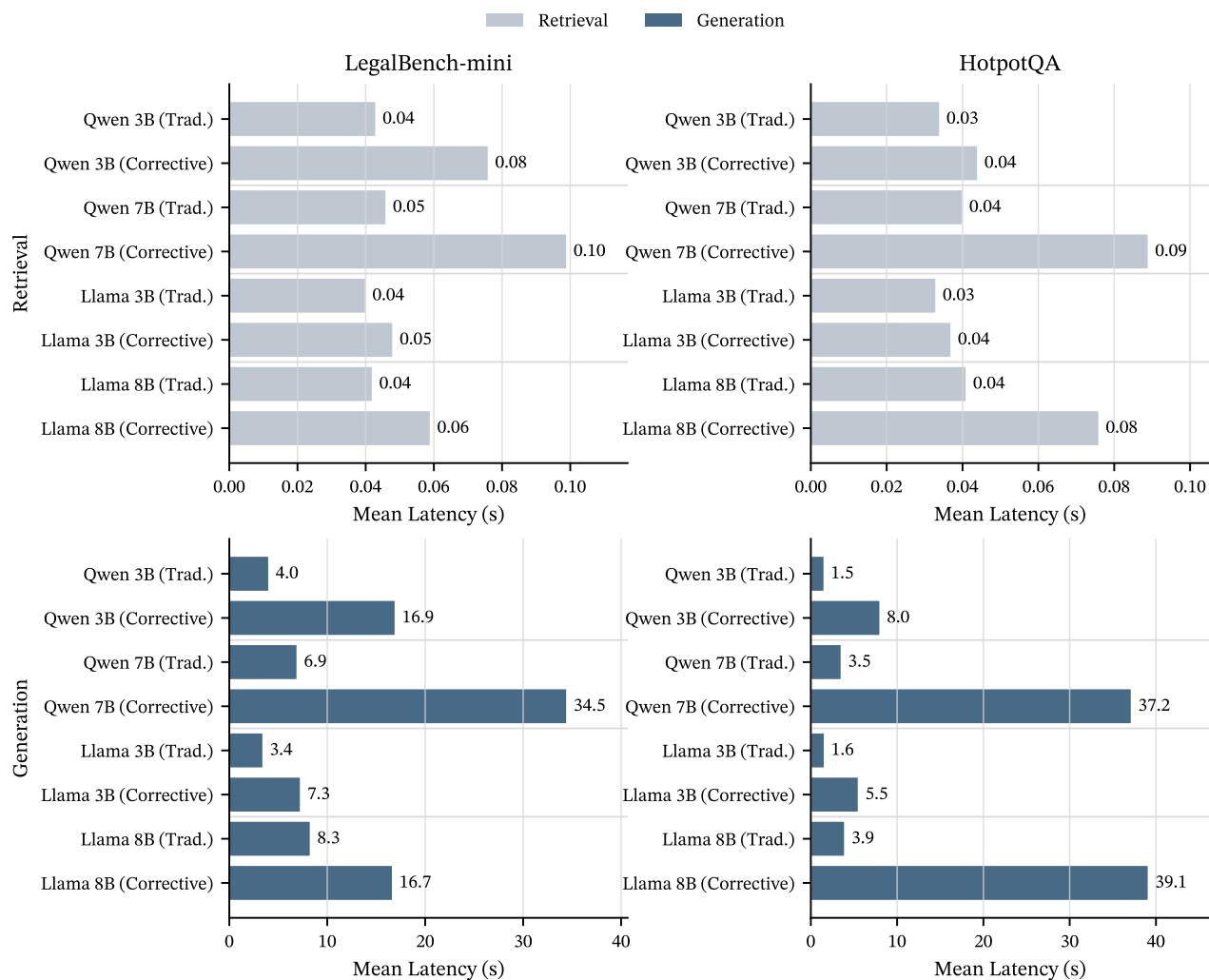


Figure E.1: Retrieval and generation latency decomposition for the controlled local runs. Generation-side latency dominates the high-cost corrective conditions, while retrieval latency remains small across conditions.

E.7 Cross-Environment Validation Details

Table E.6 reports the full metric values for the matched local and pod reruns summarized in Figure 4.4. The table is placed here because the main result is the stability pattern shown by the figure, while the detailed paired values are mainly useful for verification.

Table E.6: Cross-environment validation comparing the final local runs with matched pod reruns. Only effectiveness and corrective-behaviour metrics are shown.

Panel A: LegalBench-mini							
Condition	Local Evidence Recall	Pod Evidence Recall	Local Top- <i>k</i> Hit Rate	Pod Top- <i>k</i> Hit Rate	Local Retry Rate	Pod Retry Rate	
Qwen 3B Traditional	0.308	0.308	0.308	0.308	0.000	0.000	
Qwen 3B Corrective	0.445	0.455	0.562	0.571	0.937	0.937	
Qwen 7B Traditional	0.308	0.308	0.308	0.308	0.000	0.000	
Qwen 7B Corrective	0.519	0.518	0.647	0.649	0.955	0.955	
Llama 3B Traditional	0.308	0.308	0.308	0.308	0.000	0.000	
Llama 3B Corrective	0.308	0.308	0.308	0.308	0.000	0.000	
Llama 8B Traditional	0.308	0.308	0.308	0.308	0.000	0.000	
Llama 8B Corrective	0.308	0.308	0.308	0.308	0.000	0.000	

Panel B1: HotpotQA Quality							
Condition	Local Answer Accuracy	Pod Answer Accuracy	Local Answer F1	Pod Answer F1	Local Supporting-Title Recall	Pod Supporting-Title Recall	
Qwen 3B Traditional	0.390	0.394	0.512	0.516	0.809	0.809	
Qwen 3B Corrective	0.391	0.390	0.513	0.512	0.809	0.806	
Qwen 7B Traditional	0.451	0.450	0.578	0.577	0.809	0.809	
Qwen 7B Corrective	0.465	0.461	0.594	0.588	0.853	0.850	
Llama 3B Traditional	0.364	0.365	0.508	0.508	0.809	0.809	
Llama 3B Corrective	0.364	0.368	0.508	0.509	0.809	0.809	
Llama 8B Traditional	0.440	0.435	0.573	0.570	0.809	0.809	
Llama 8B Corrective	0.443	0.442	0.576	0.578	0.826	0.822	

Panel B2: HotpotQA Behaviour		
Condition	Local Retry Rate	Pod Retry Rate
Qwen 3B Traditional	0.000	0.000
Qwen 3B Corrective	0.210	0.193
Qwen 7B Traditional	0.000	0.000
Qwen 7B Corrective	0.863	0.855
Llama 3B Traditional	0.000	0.000
Llama 3B Corrective	0.002	0.001
Llama 8B Traditional	0.000	0.000
Llama 8B Corrective	0.862	0.861